

Deep nets and backprop

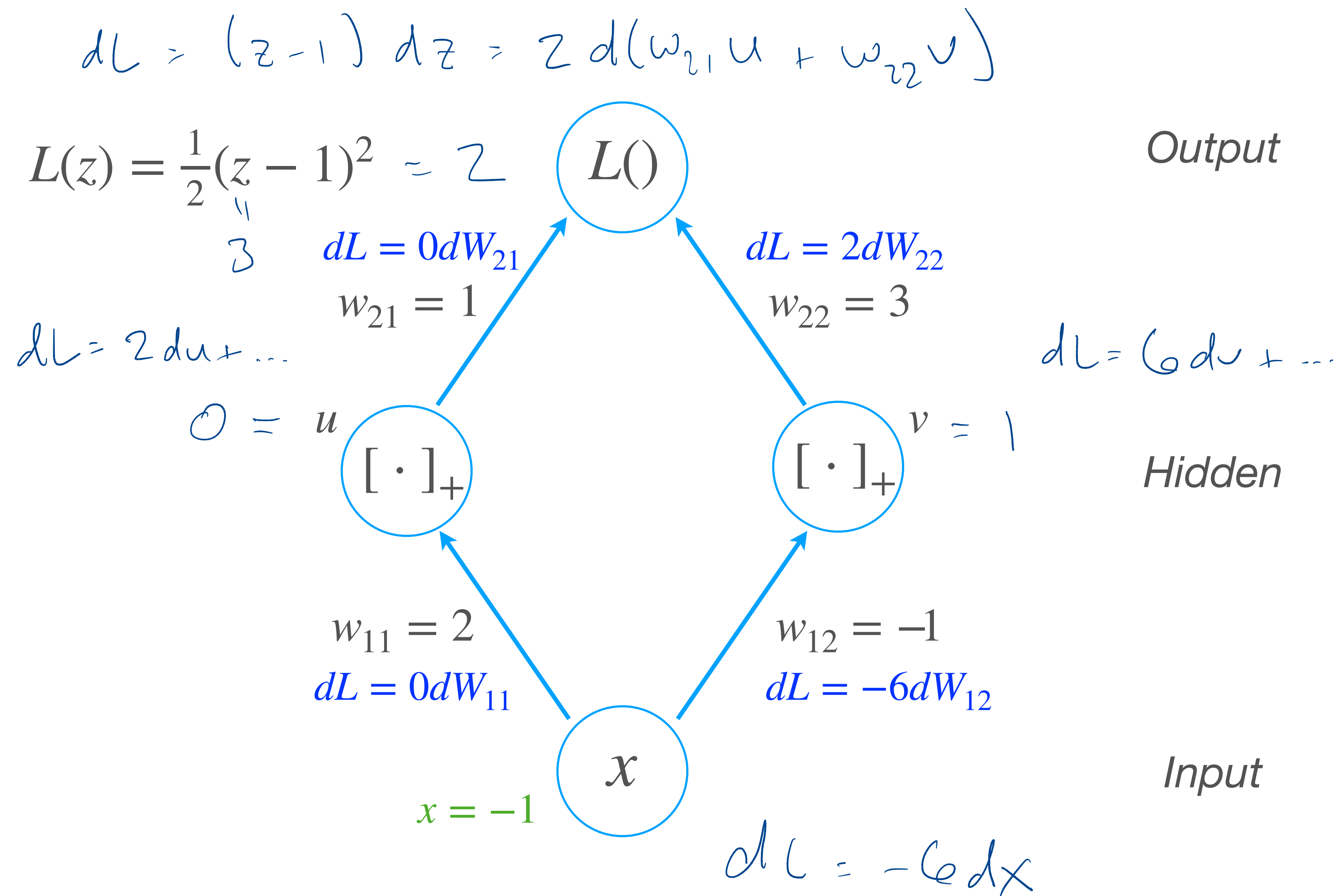


10-701 Introduction to Machine Learning
Geoff Gordon

Try another example

$$v = [\omega_{12} x]_+$$

$$dv = 1 d(\omega_{12} x)$$



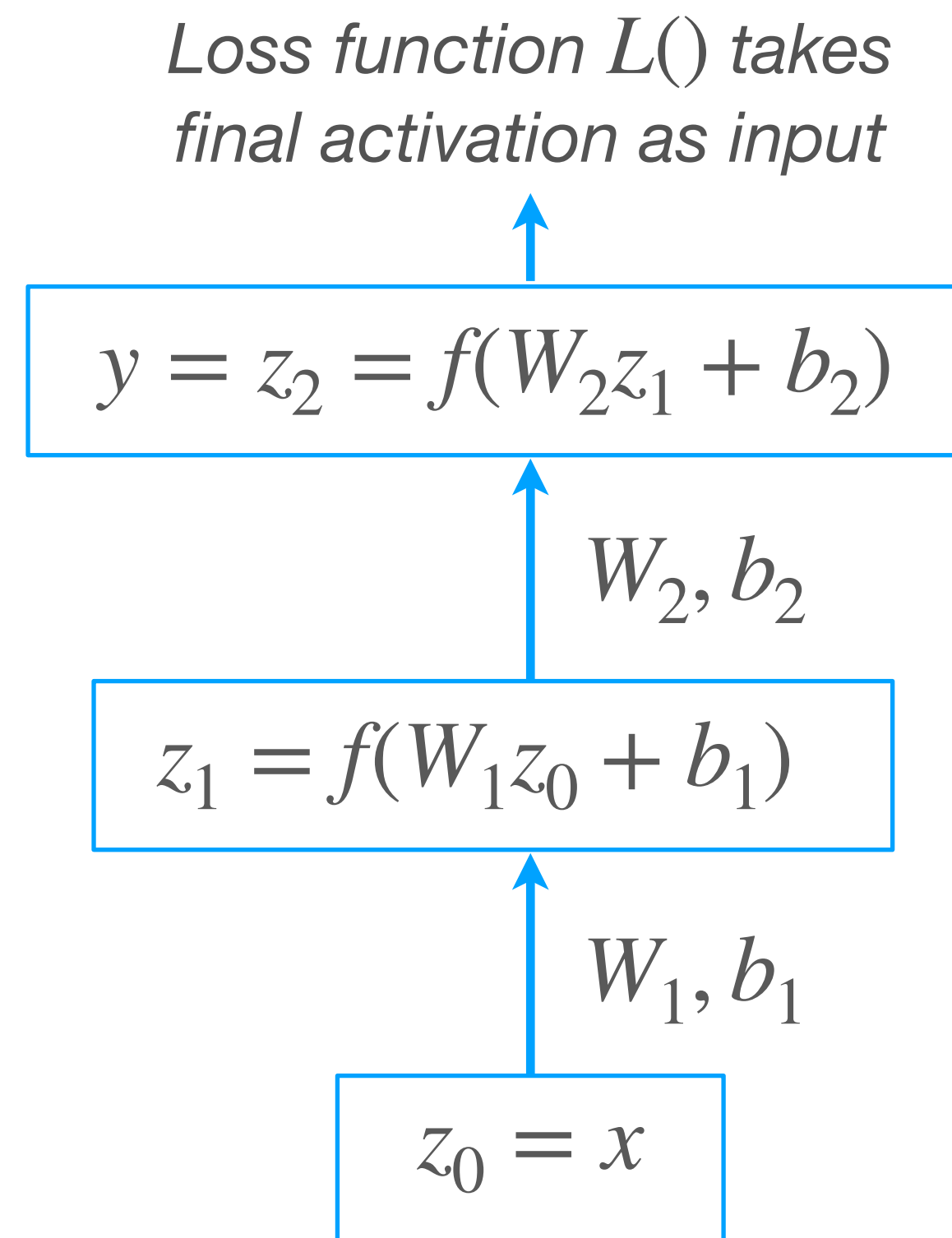
Backprop

- Forward and backward passes are ***automatable***
 - ▶ a few simple operations we do over and over
 - ▶ linear function, activation function, activation derivative, chain rule
- Resulting algorithm is ***backward propagation*** or ***backprop***
- Responsible for neural networks' initial rise to popularity

Backprop setup

This kind of network is called a **multi-layer perceptron** or *MLP*

often have a different activation at last layer (e.g., identity)



$$z_i, b_i \in \mathbb{R}^{d_i}$$
$$W_i \in \mathbb{R}^{d_i \times d_{i-1}}$$

activations z_i , activation function f (componentwise), weights W_i , biases b_i (weight for constant feature)

- Suppose forward pass is: $z_i = f(W_i z_{i-1} + b_i)$
 - ▶ that is, set $z_0 = x$, iterate $i = 1, 2, \dots, K$ computing **pre-activation** $u_i = W_i z_{i-1} + b_i$ and activation $z_i = f(u_i)$
 - ▶ read off output $y = z_K$ and pass as input to $L()$

Backprop algorithm

- Goal: derivatives of loss L wrt weights W_i and biases b_i
 - ▶ as a byproduct, also wrt activation vectors z_i
- Loop invariant: we know $dL = \langle r_i, dz_i \rangle$
 - ▶ vector r_i is one of the activation derivatives
 - ▶ initialize at layer K w/ gradient of loss function (wrt its input, the final activation z_K)
- At each layer from K down to 1:
 - ▶ scale by derivative of activation f : define $v_i = f'(u_i) \circ r_i$
 - ▶ dL/db_i is just v_i
 - ▶ dL/dW_i is outer product $v_i z_{i-1}^\top$
- Update invariant: $r_{i-1} = W_i^\top v_i$

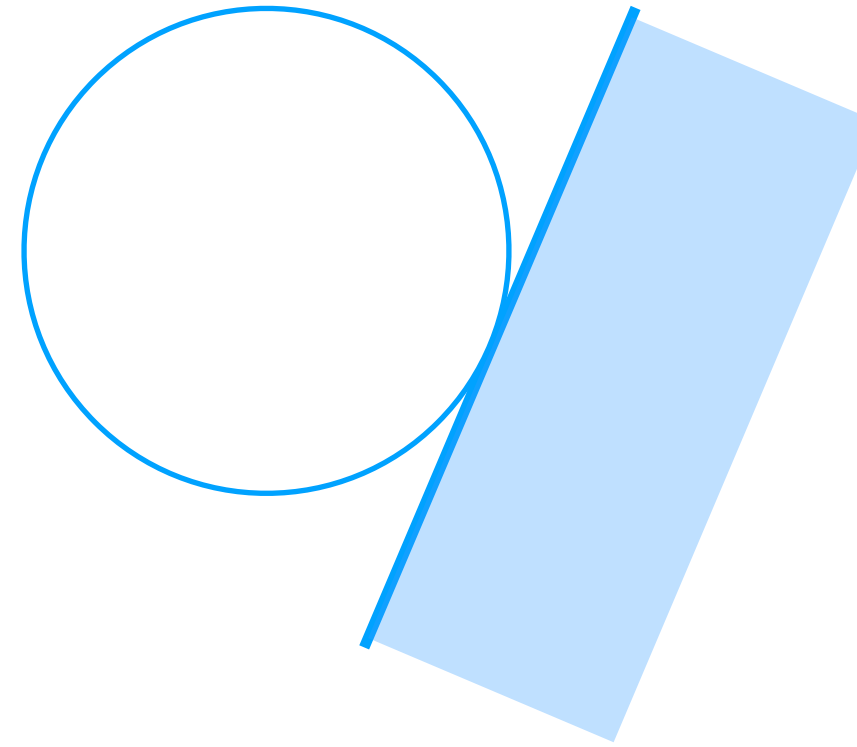
Fitting the model by SGD

- Sample an (x, y)
 - ▶ forward pass to compute $f_w(x)$ using weights w , saving intermediate values (hidden activations)
 - ▶ find loss, e.g. $L = \frac{1}{2}(y - f_w(x))^2$
 - ▶ backward pass to compute dL as a linear function of dw
- Take an optimizer step on w : e.g., step downhill using derivative (coefficient of dw)
- Or, for each (x, y) in minibatch
 - ▶ do above, accumulate derivatives (coefficients of dw)
 - ▶ step using sum or mean of derivatives

Why depth?

- ML needs two things to work:
 - ▶ expressivity: model has to be able to represent (something close to) the thing we want to learn
 - ▶ simplicity: need a regularizer or complexity measure that rules out bad-yet-consistent hypotheses — depth, width, work, norm of weights, ...
- Tension:
 - ▶ expressivity: want to learn e.g. arbitrary Python code
 - ▶ simplicity: there are a *lot* of Python programs to search through, and we don't know how to rule out bad ones
- Depth seems to facilitate a tradeoff: simple families of deep programs contain useful functions at low complexity levels

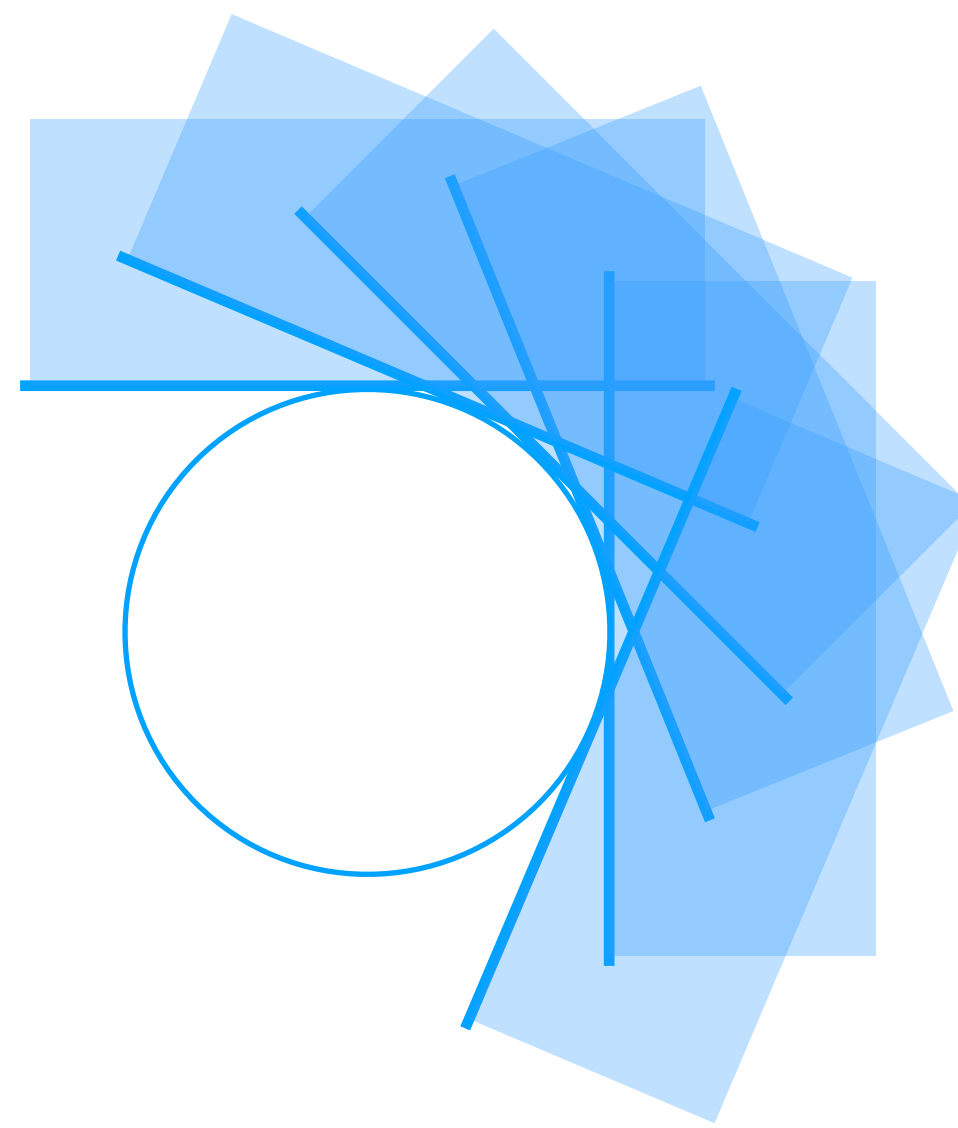
Expressivity



For example, we can approximate a circular decision boundary by summing many step functions and thresholding

- Neural networks are ***universal approximators***: even with just one hidden layer, they can fit any continuous function on a compact set arbitrarily well [*Cybenko, 1989; Hornik, 1991*]
- But, might need large or huge numbers of hidden units:
 - ▶ if function is less smooth, if we want approximation in $\| \cdot \|_{\infty}$, if we require bounded weights, ...

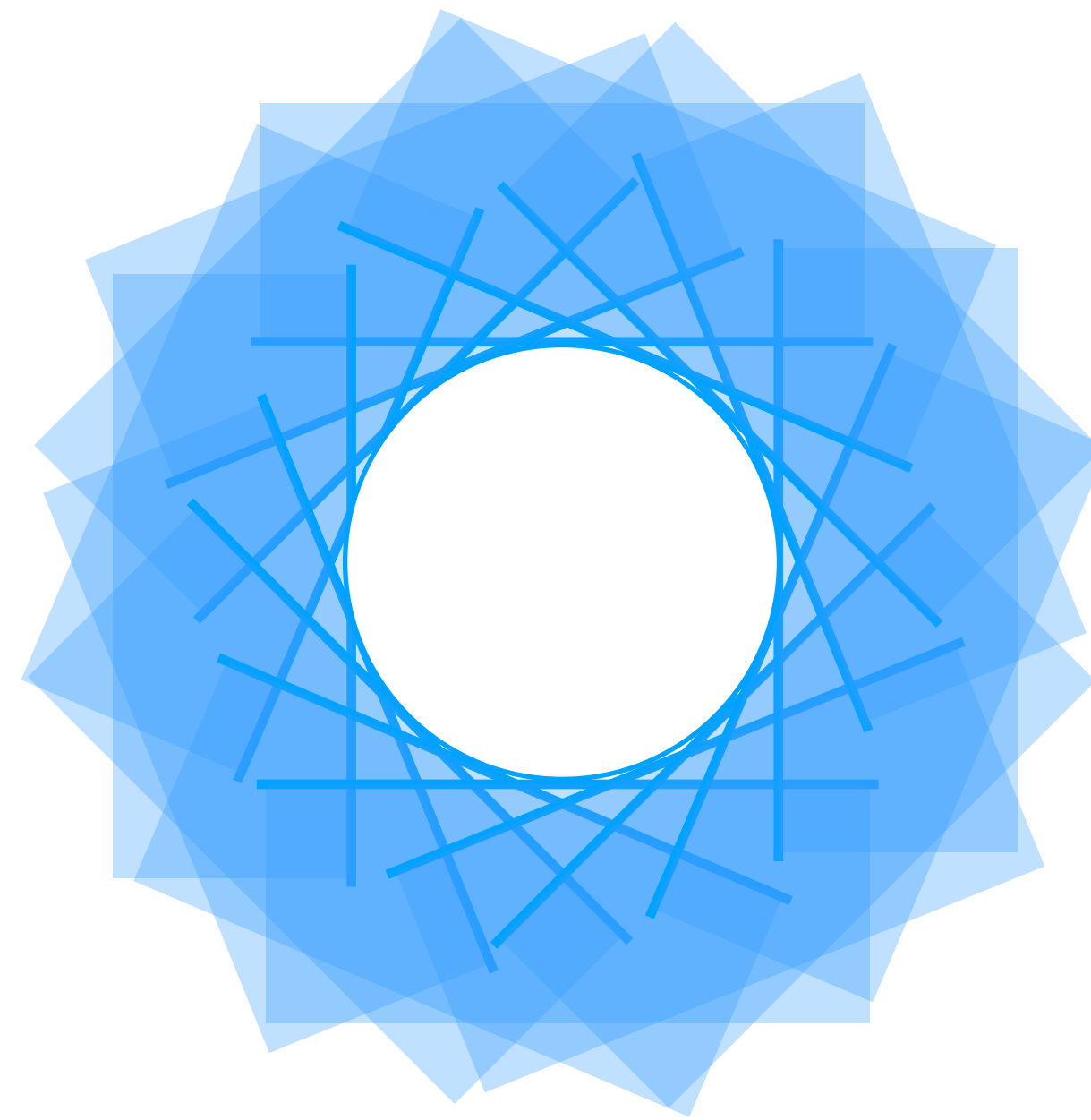
Expressivity



For example, we can approximate a circular decision boundary by summing many step functions and thresholding

- Neural networks are ***universal approximators***: even with just one hidden layer, they can fit any continuous function on a compact set arbitrarily well [*Cybenko, 1989; Hornik, 1991*]
- But, might need large or huge numbers of hidden units:
 - ▶ if function is less smooth, if we want approximation in $\|\cdot\|_\infty$, if we require bounded weights, ...

Expressivity



For example, we can approximate a circular decision boundary by summing many step functions and thresholding

- Neural networks are ***universal approximators***: even with just one hidden layer, they can fit any continuous function on a compact set arbitrarily well [*Cybenko, 1989; Hornik, 1991*]
- But, might need large or huge numbers of hidden units:
 - ▶ if function is less smooth, if we want approximation in $\| \cdot \|_{\infty}$, if we require bounded weights, ...

Expressivity, part II

- Depth provably makes a big difference
- For any k , compare programs that are
 - ▶ *shallow*: depth $\Theta(k)$ steps, or
 - ▶ *deep*: depth $\Theta(k^3)$ steps
- **Thm** [Telgarsky, 2016]: there are (many) functions that we can implement with *constant* width in deep programs but require *exponential* width $\Omega(2^k)$ even to approximate with shallow programs
 - ▶ step: can be quite general, e.g., piecewise polynomial fn of previous outputs

Why are classical models often shallow?

- Optimization
 - ▶ not obvious how to find good model in deep model class (need solutions to regularization, local minima, vanishing/exploding gradients, sharpness, ...)
- Generalization
 - ▶ deep model class typically has very high VC dimension / Rademacher complexity / anything else we can measure
 - ▶ need to co-design architecture and optimizer to get implicit regularization
- These used to be big problems
 - ▶ when I took 10-701:
“don’t try to use SGD to train a model with depth > 2 or 3”
- Spoiler: not any more

Fitting the model, autodiff version

Note: `loss` is a scalar

For vector, matrix, or tensor outputs: PyTorch can compute inner product of gradient with any constant

- Use `Tensor` class for all arrays
 - ▶ keeps the “paper trail” of who depends on whom
- For each example in minibatch
 - ▶ run code that starts w/ (x, y) and ends w/ `loss += ...`
- Call `loss.backward()` to run autodiff
- Take an optimizer step (e.g., SGD) using accumulated derivative (pay attention to learning rate scaling w/ B)
 - ▶ we tell optimizer on init which `Tensors` hold parameters
- Clear derivative storage (e.g., SGD optimizer has `zero_grad()` method, as do other optimizers and also classes for models)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Make it deeper

- Now that we have autodiff, no problem differentiating a 50-layer network!
- Let's try it:
 - ▶ $x, z_i, w_i, b_i \in \mathbb{R}$
 - ▶ $z_0 = x$
 - ▶ $z_i = \sigma(w_i z_{i-1} + b_i),$
i=1..50
 - ▶ initialize
 $w_i, b_i \sim N(0, 0.1^2)$
- Forward pass:
 $z_{50} = 0.5167$
- Backward pass: $\rightarrow$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Make it deeper

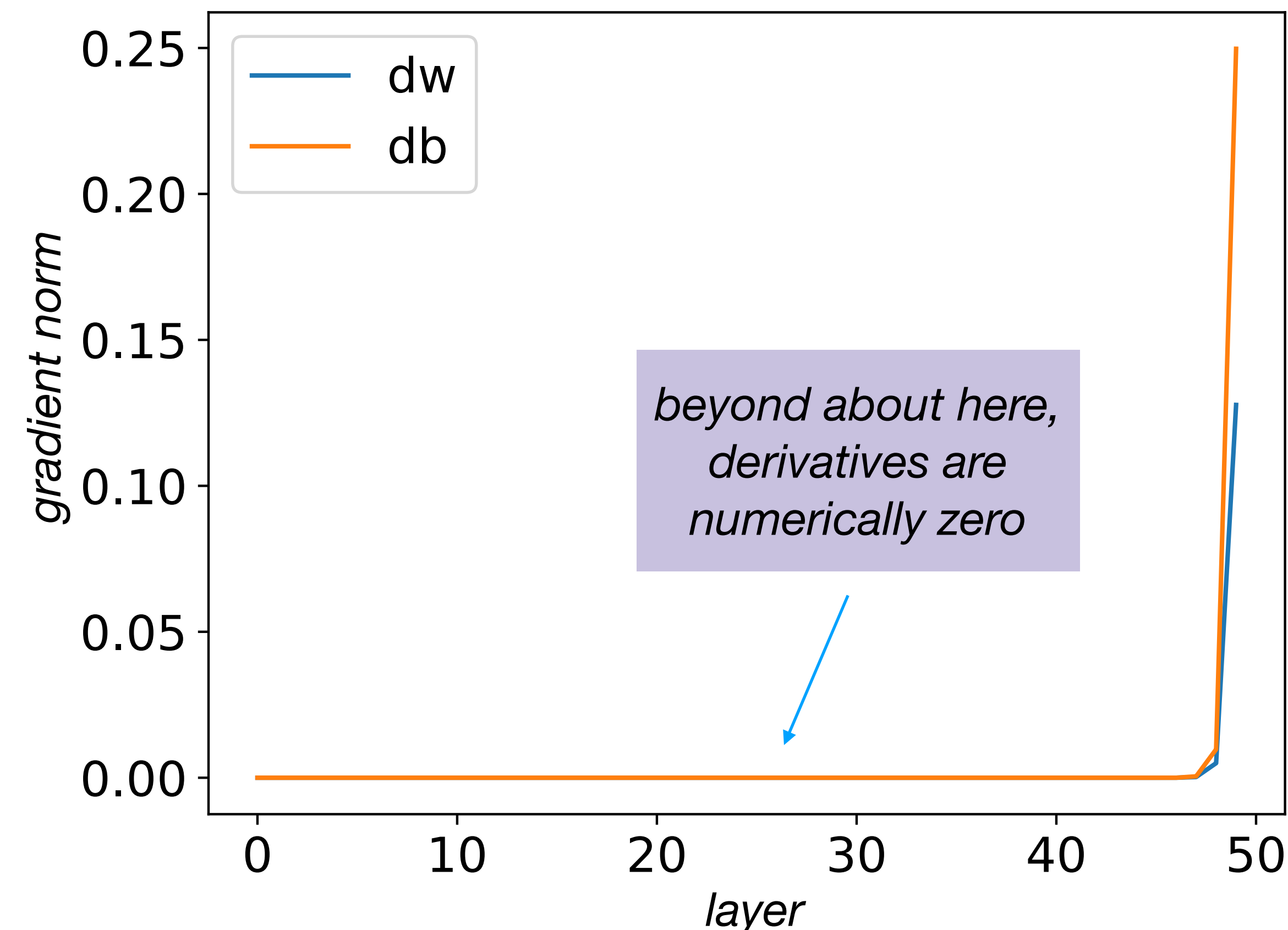
- Now that we have autodiff, no problem differentiating a 50-layer network!

- Let's try it:

- ▶ $x, z_i, w_i, b_i \in \mathbb{R}$
- ▶ $z_0 = x$
- ▶ $z_i = \sigma(w_i z_{i-1} + b_i), i=1..50$
- ▶ initialize $w_i, b_i \sim N(0, 0.1^2)$

- Forward pass:
 $z_{50} = 0.5167$

- Backward pass: →



Is this a problem?

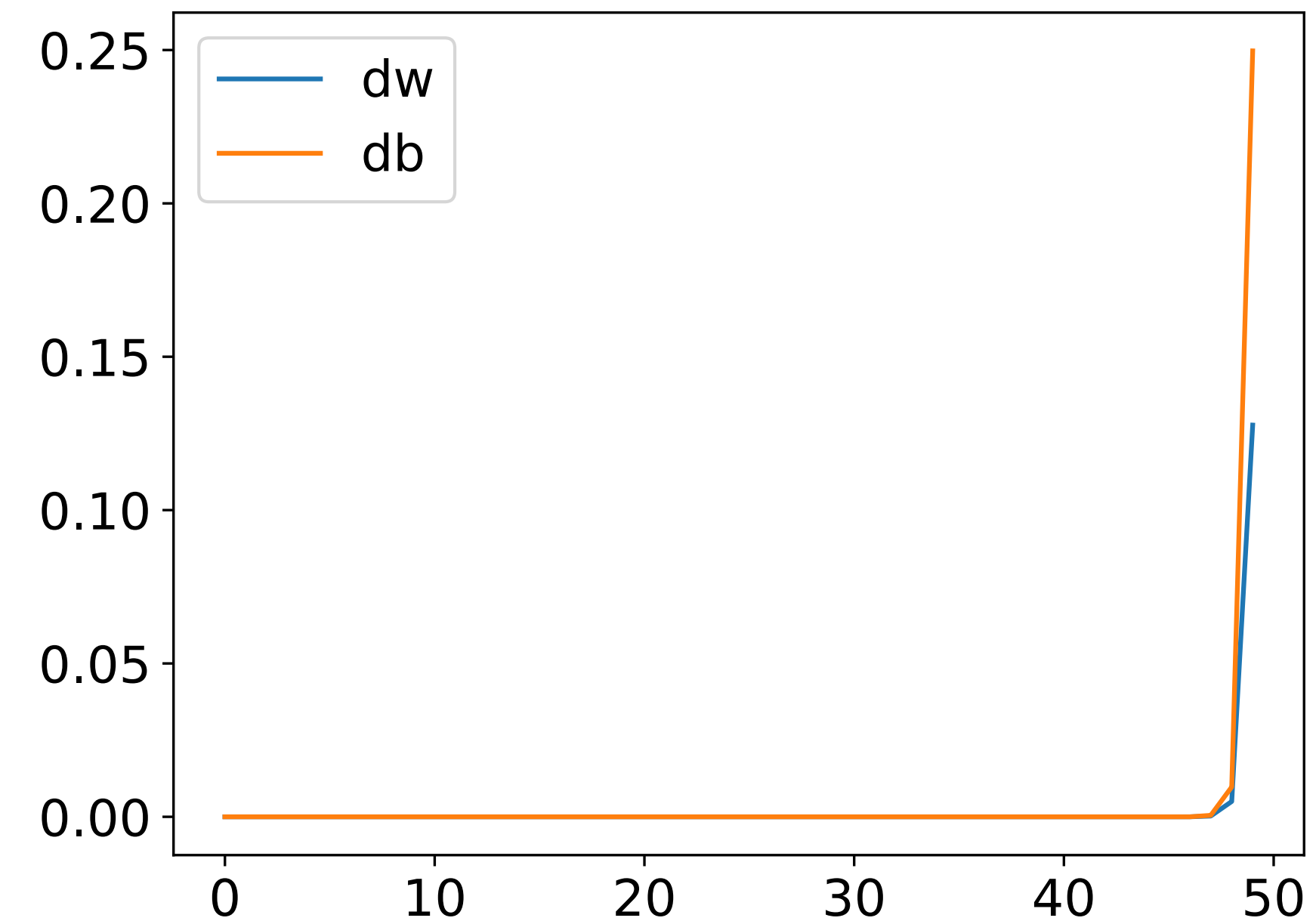
- In SGD: effectively no updates to w_i, b_i for $i \leq 45$, and truly zero updates for $i \leq 30$
 - ▶ so, we aren't really using a deep model: only last few layers are learnable
 - ▶ worse: in forward pass, $z_{30} \approx 0.5$ no matter what x is
 - ▶ $dz_{30} = 0 dx$ numerically
 - ▶ i.e., we ignore the input

Why does this happen?

- Run backprop:

$$\begin{aligned} dz_{50} &= \sigma'(w_{50}z_{49} + b_{50})w_{50}dz_{49} \\ &= \sigma'(w_{50}z_{49} + b_{50})w_{50}\sigma'(w_{49}z_{48} + b_{49})w_{49}dz_{48} \\ &= \dots \end{aligned}$$

- all σ' terms $\approx \frac{1}{4}$, all w_i terms ≈ 0.1 in magnitude
 - ▶ exponential decay: called *vanishing gradients*



Exploding gradients

- Can get a related problem if we initialize to *large* weights
- Keep multiplying gradient by $w_i \rightarrow$ exponential growth
- But growth can be self-limiting:
 - ▶ e.g., with sigmoid nonlinearity: $\sigma'(z) \approx 0$ when $|z|$ large
- So, paradoxically, explosion can also lead to near-zero gradients
 - ▶ which then blow up again and zero out again, chaotically

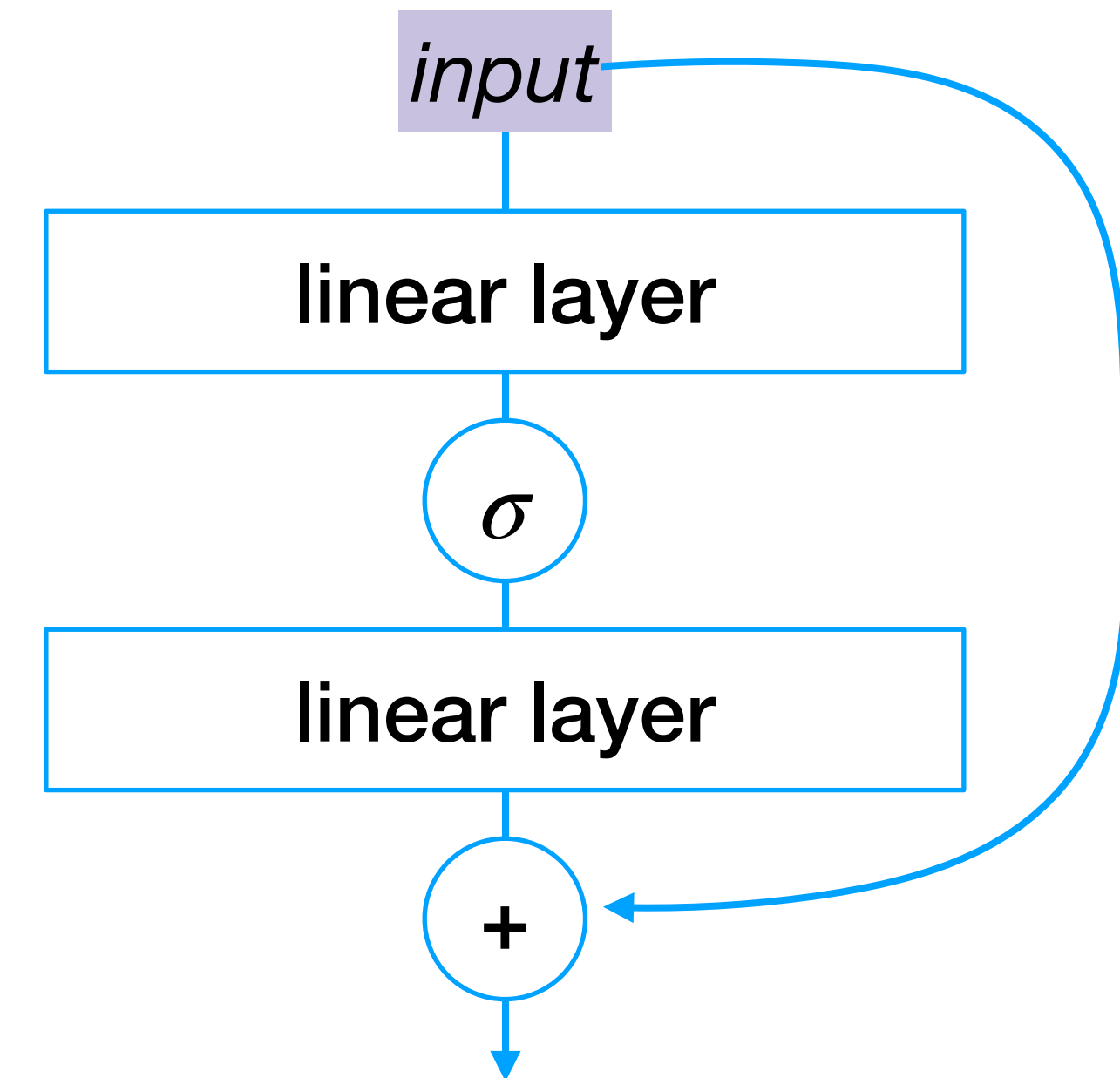
Just right

*sometimes called
edge of chaos: it
skates the edge
between vanishing
gradients and
chaotic explosion*

- What if we initialize to moderate-sized weights?
- Helps a lot:
 - ▶ generally enough for moderate-depth networks (say, 10)
 - ▶ beyond that, hard to control for activation gradient
- In more detail:
 - ▶ if we know approximate variance of vector z_t at layer t
 - ▶ and we choose variance V of elements of w_t
 - ▶ then we can calculate $\text{var}(z_t \cdot w_t)$
 - ▶ and multiply by an estimate of squared activation gradient to get variance of z_{t+1}
 - ▶ so adjusting V can keep both activations and gradients from growing or shrinking too much
- PyTorch default is similar but skips activation gradient (also supports different schemes that include it)

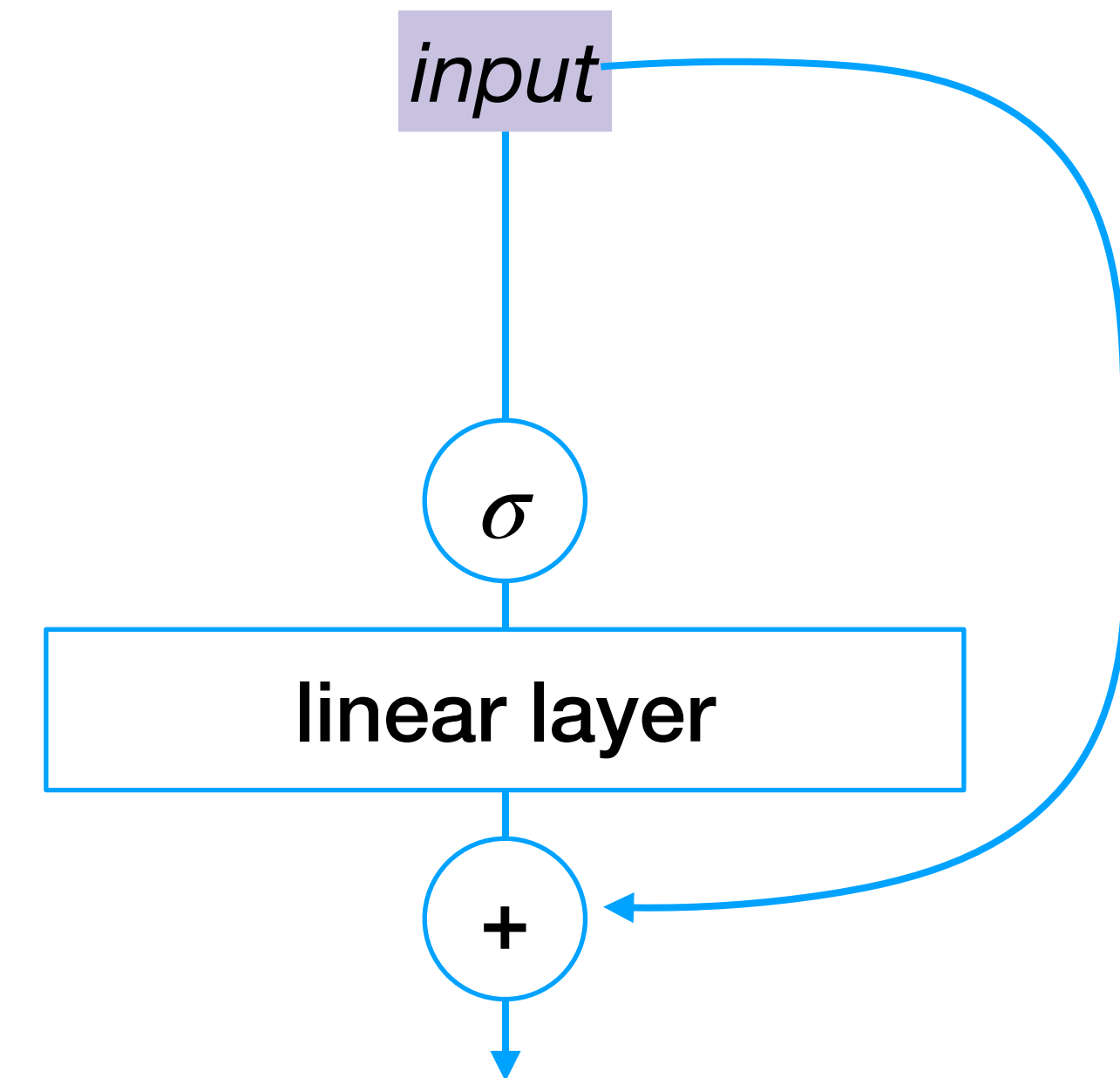
Better yet: residual connection

- Change the arrangement of layers:
 - ▶ $z_i = z_{i-1} + v_i \sigma(w_i z_{i-1} + c_i) + b_i$
 - ▶ can be any nonlinearity
 - ▶ weight matrices v_i, w_i
 - ▶ bias vectors b_i, c_i
- Sum node means we learn z_i as an *update* to z_{i-1}
- Second linear layer lets us learn sign of update, even if nonlinearity is always positive like $\sigma(z)$ or $[z]_+$
 - ▶ in fact, OK to skip first linear layer



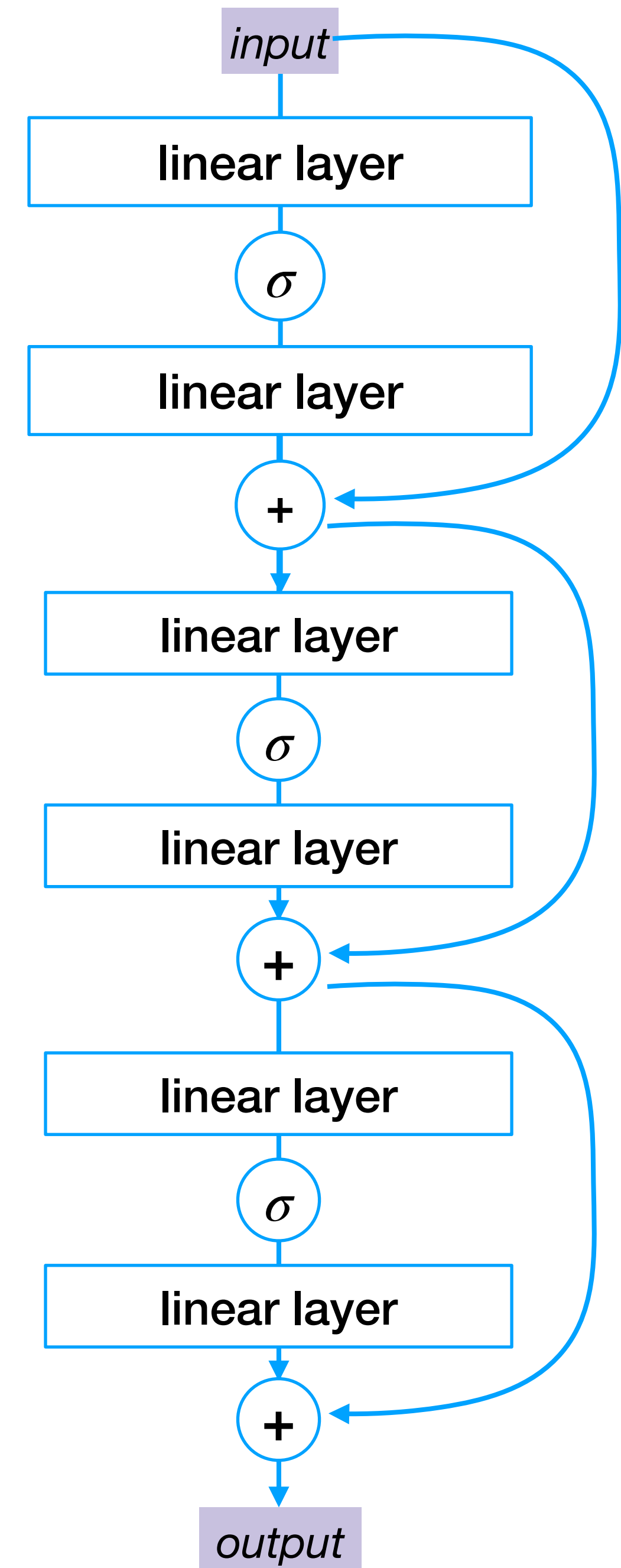
Better yet: residual connection

- Change the arrangement of layers:
 - ▶ $z_i = z_{i-1} + v_i \sigma(w_i z_{i-1} + c_i) + b_i$
 - ▶ can be any nonlinearity
 - ▶ weight matrices v_i, w_i
 - ▶ bias vectors b_i, c_i
- Sum node means we learn z_i as an *update* to z_{i-1}
- Second linear layer lets us learn sign of update, even if nonlinearity is always positive like $\sigma(z)$ or $[z]_+$
 - ▶ in fact, OK to skip first linear layer



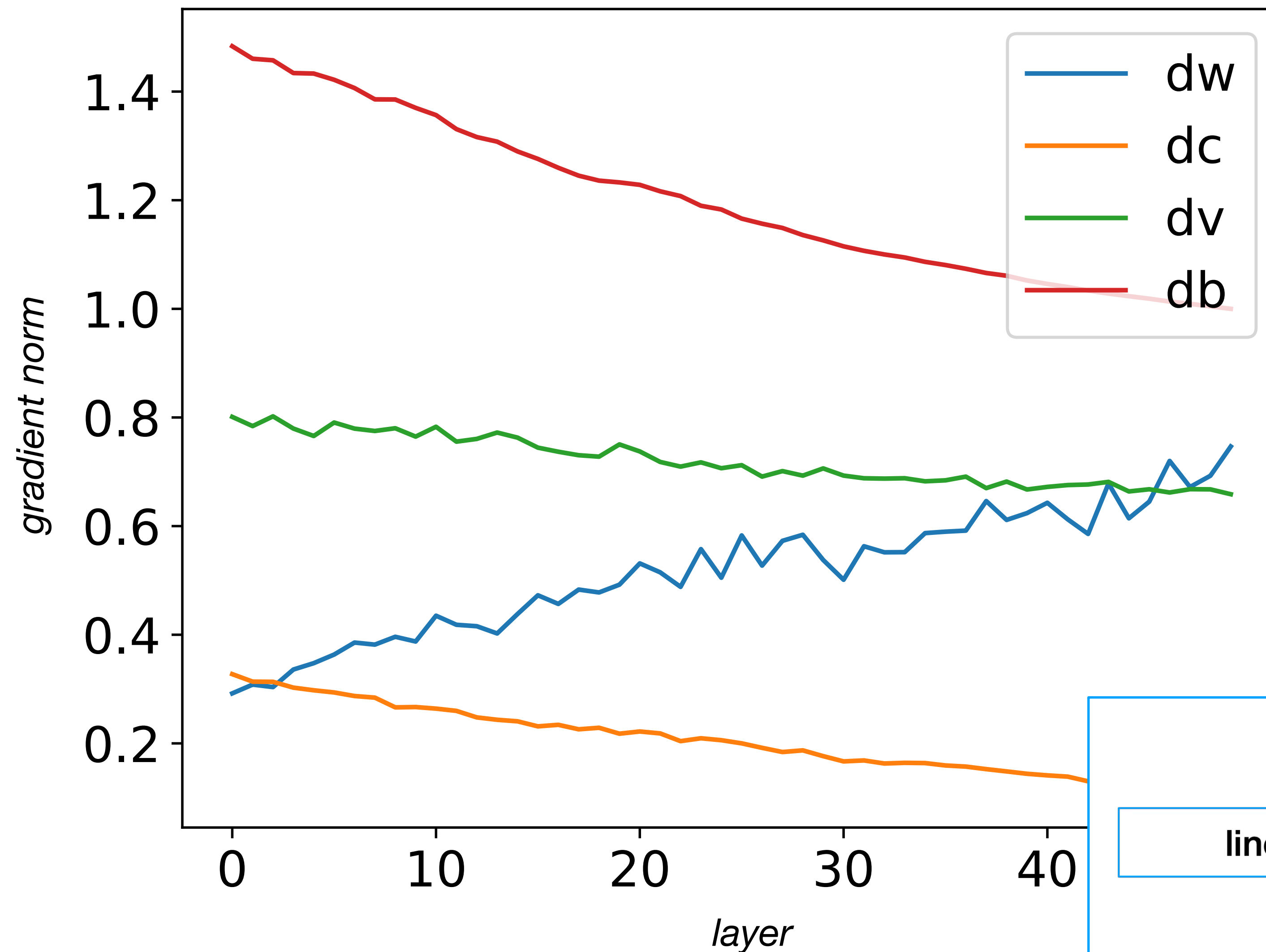
Residual networks

- Can stack residual blocks like any layer
 - ▶ appears in famous architectures like ResNet, Transformer
- Why does it prevent vanishing?
 - ▶ for each block, two paths for gradient to propagate back
 - ▶ one (the residual path) has factor 1
- Edge-of-chaos weight scaling is no longer necessary (other considerations, see below)

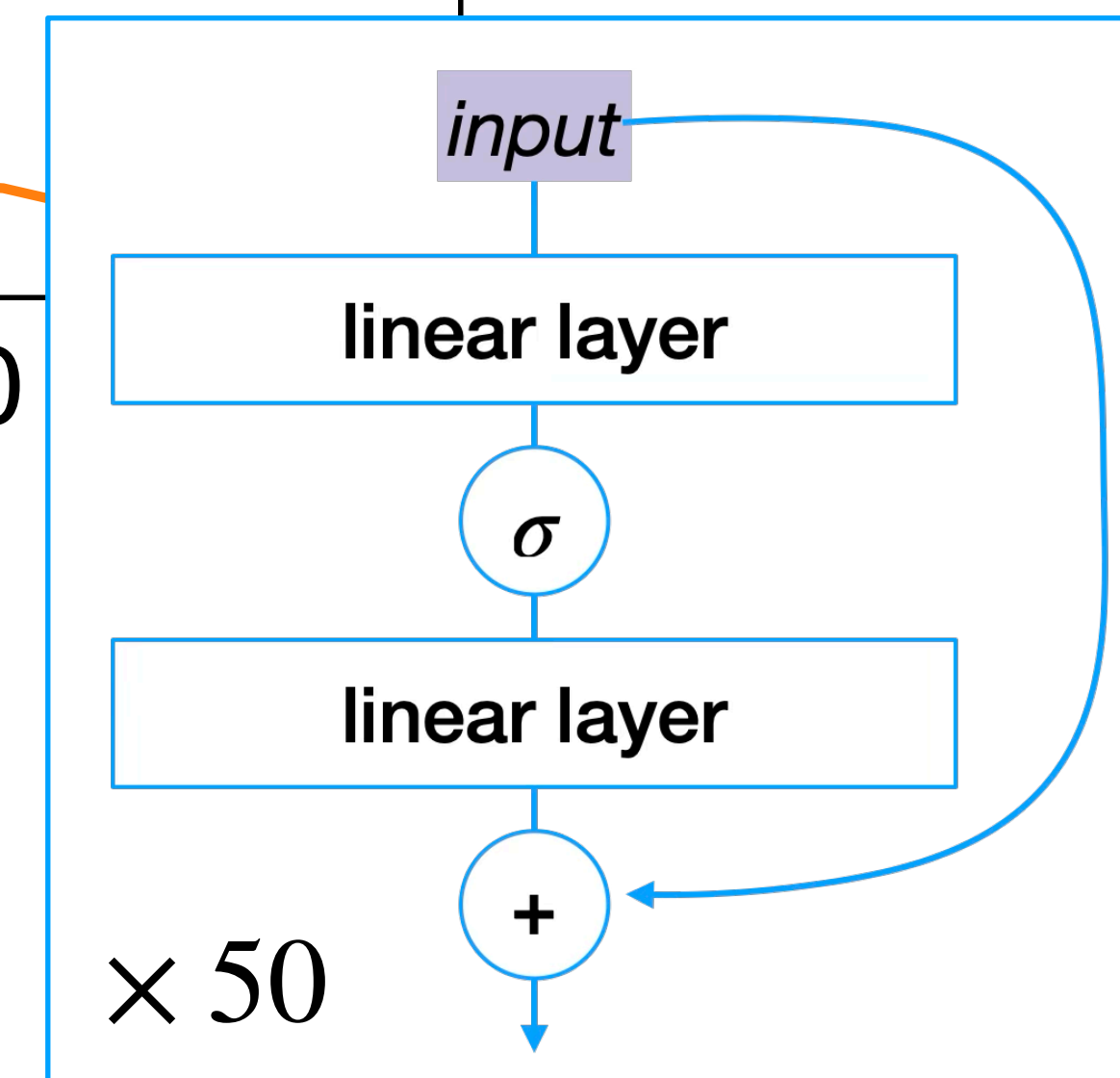


*This is an
architecture
diagram*

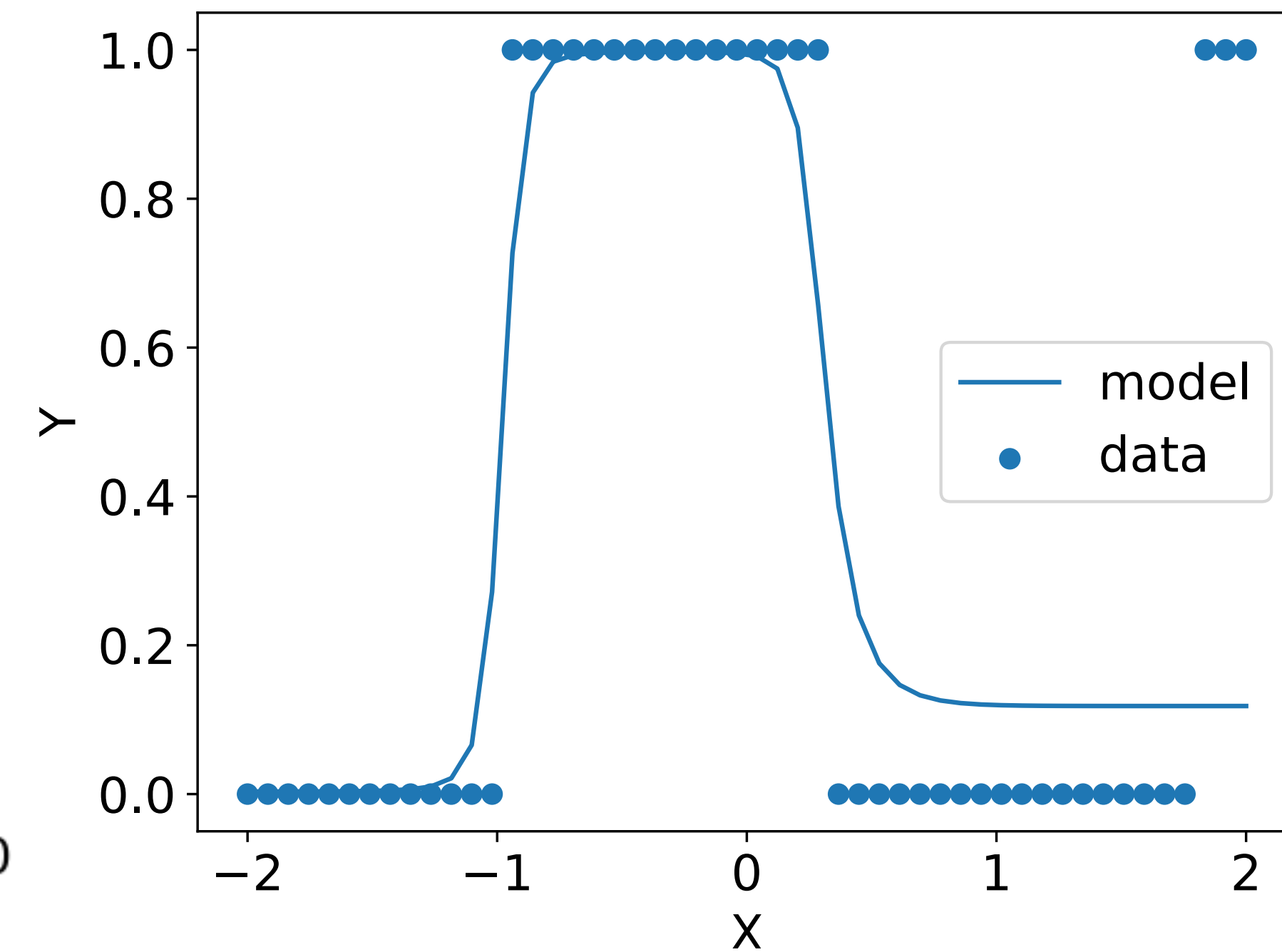
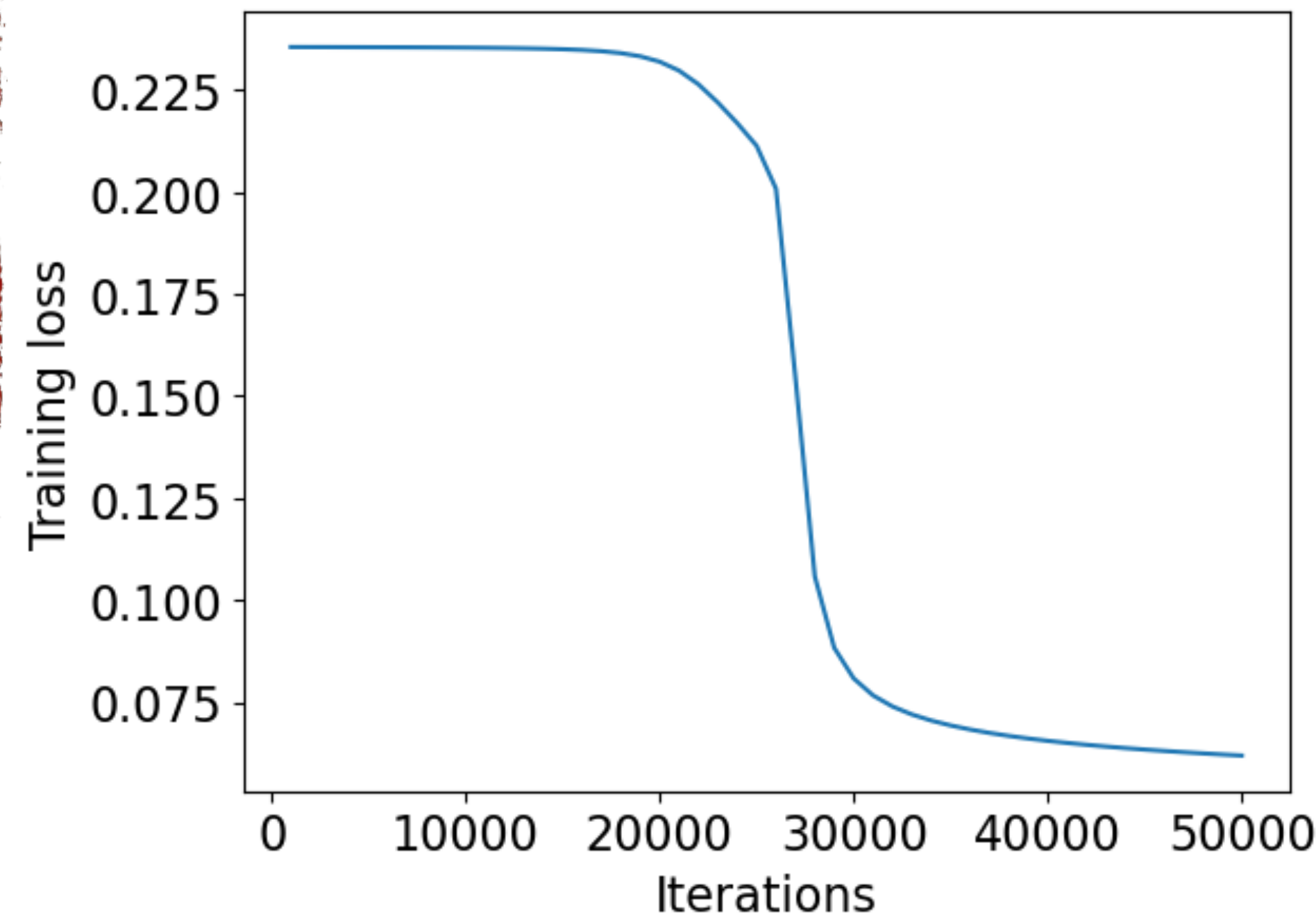
Gradient norm for residual blocks



● 50 stacked residual blocks

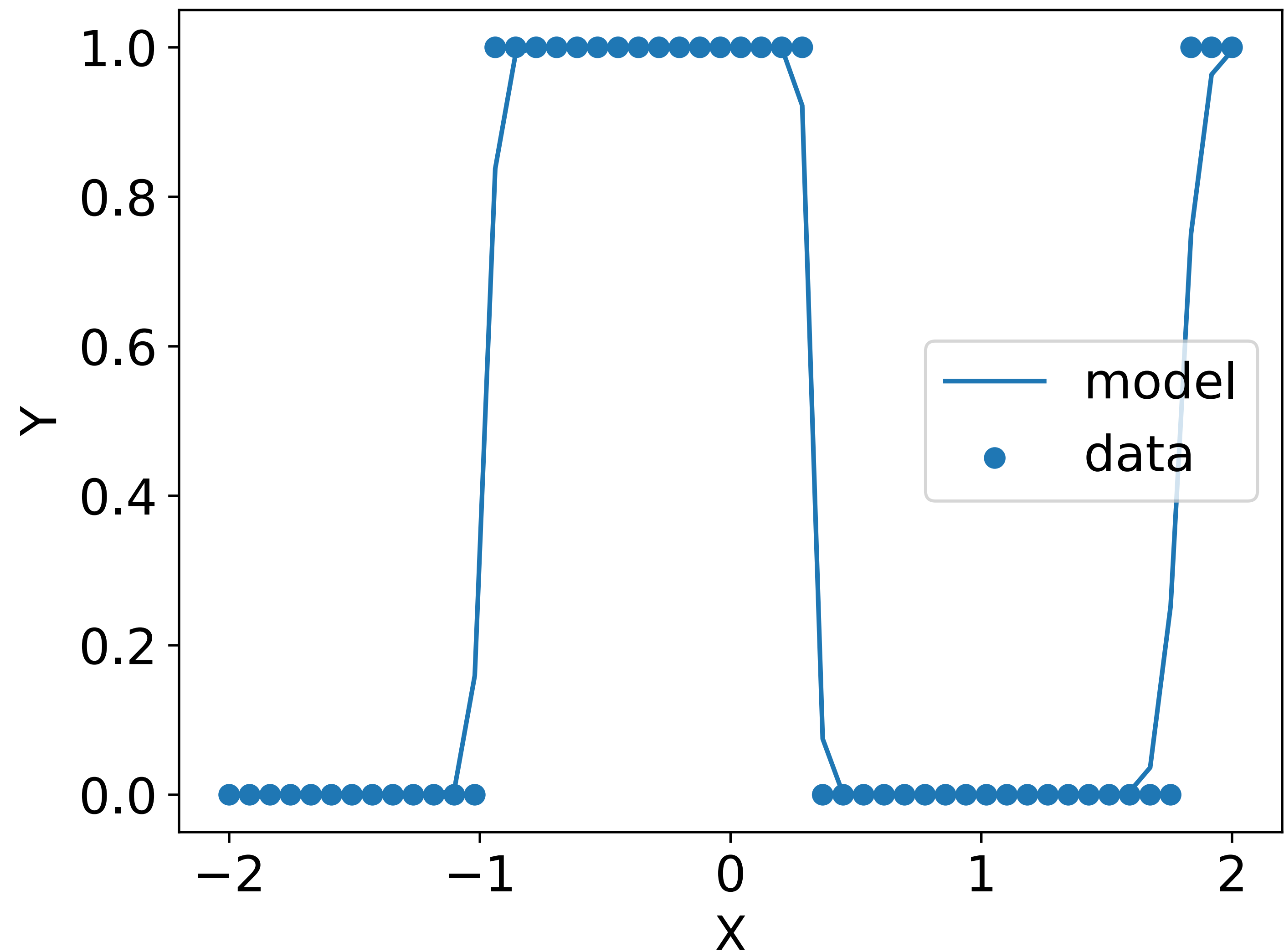


Bigger models seem to make GD/SGD work better



- Simple dataset of 50 points, 3-layer network
 - ▶ enough width to fit training data exactly
- GD, learning rate = 0.01, $\min \frac{1}{N} \sum_i (y_i - f_{W_{1:3}, b_{1:3}}(x_i))^2$
 - ▶ interestingly: GD doesn't find exact solution

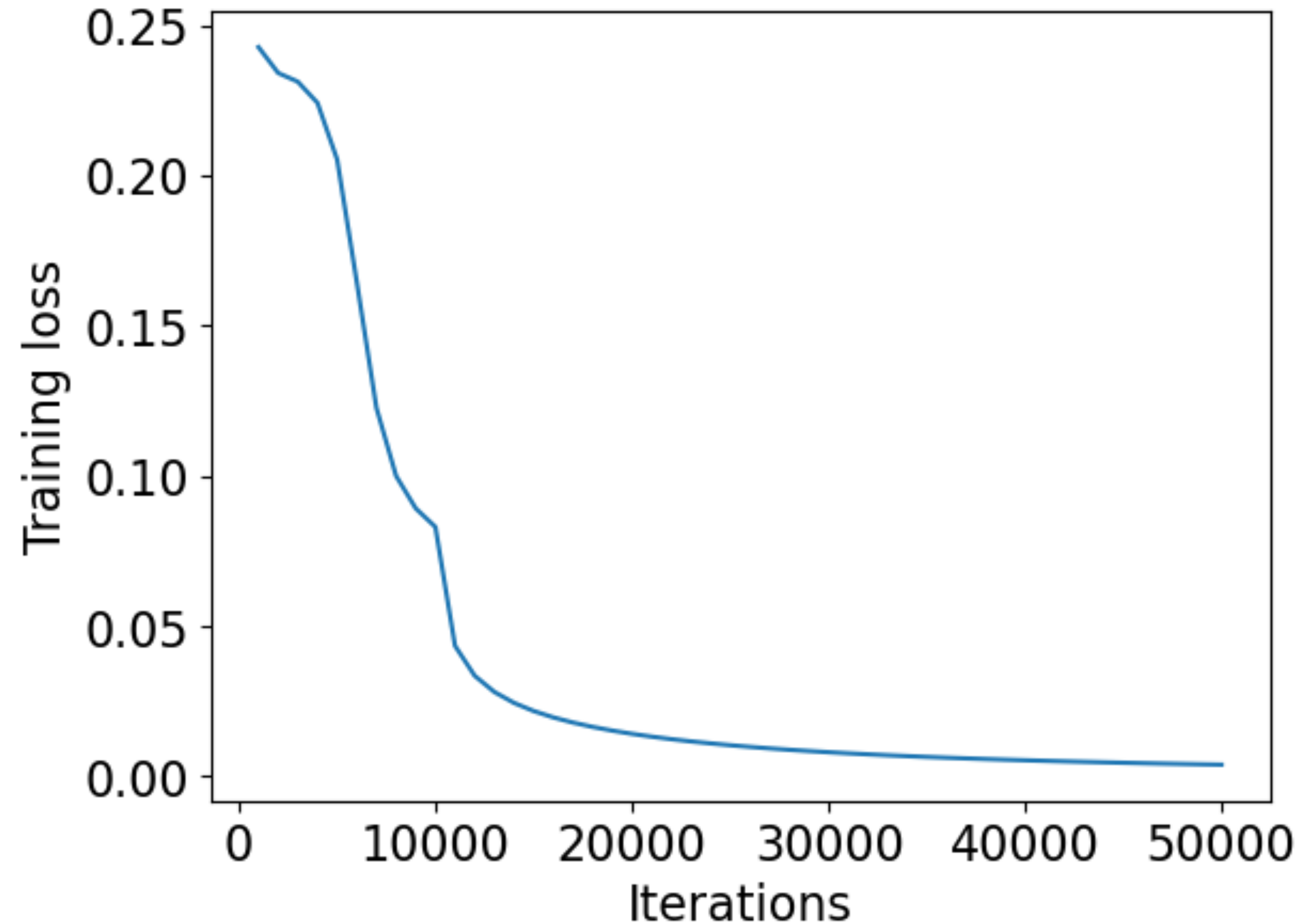
***Wider
network
finds a
better
optimum***



- Multiply width by 100 → perfect on training data

not obvious why

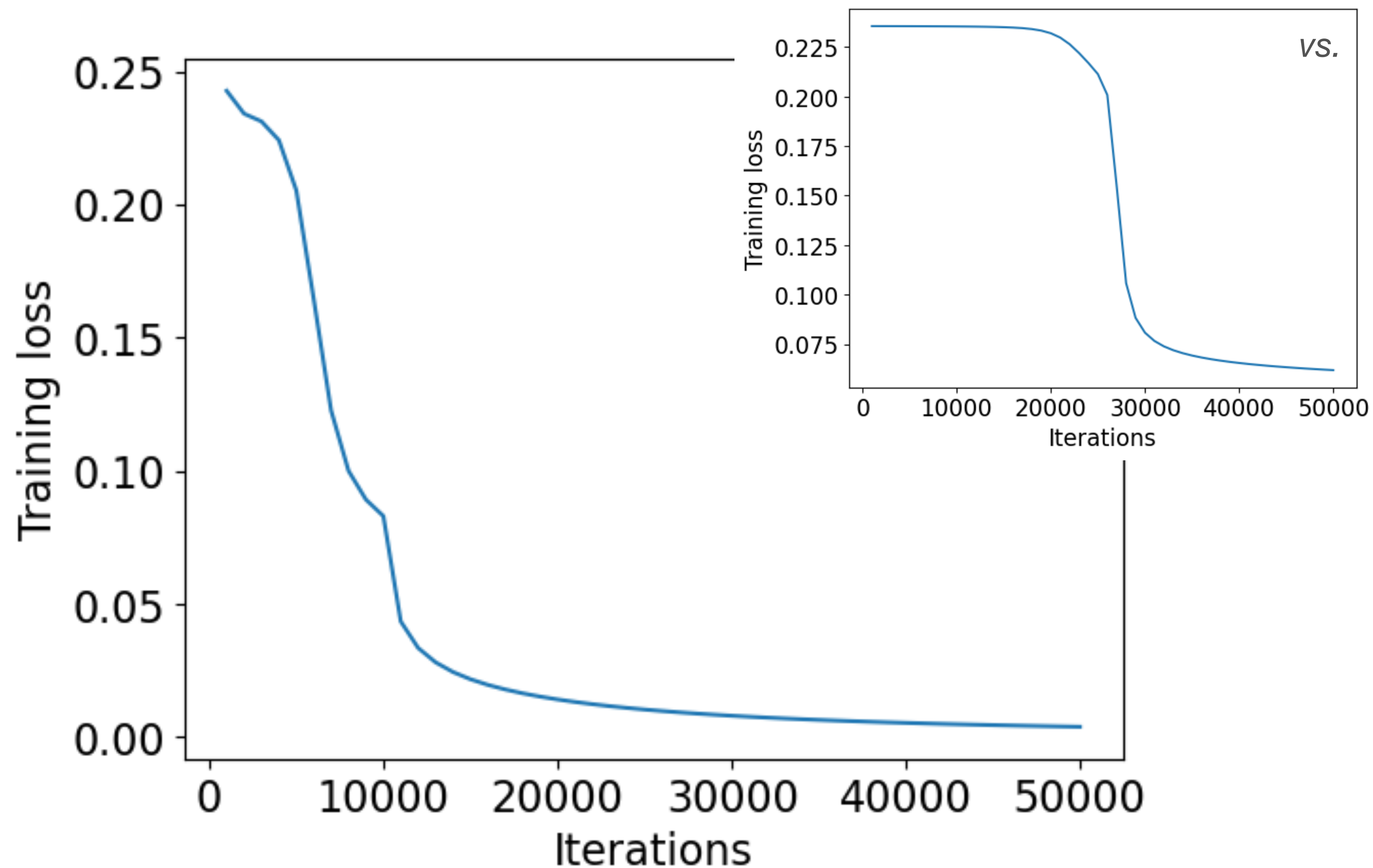
***Wider
optimizes
faster***



- Not just better endpoint, but fewer epochs to train

not obvious why

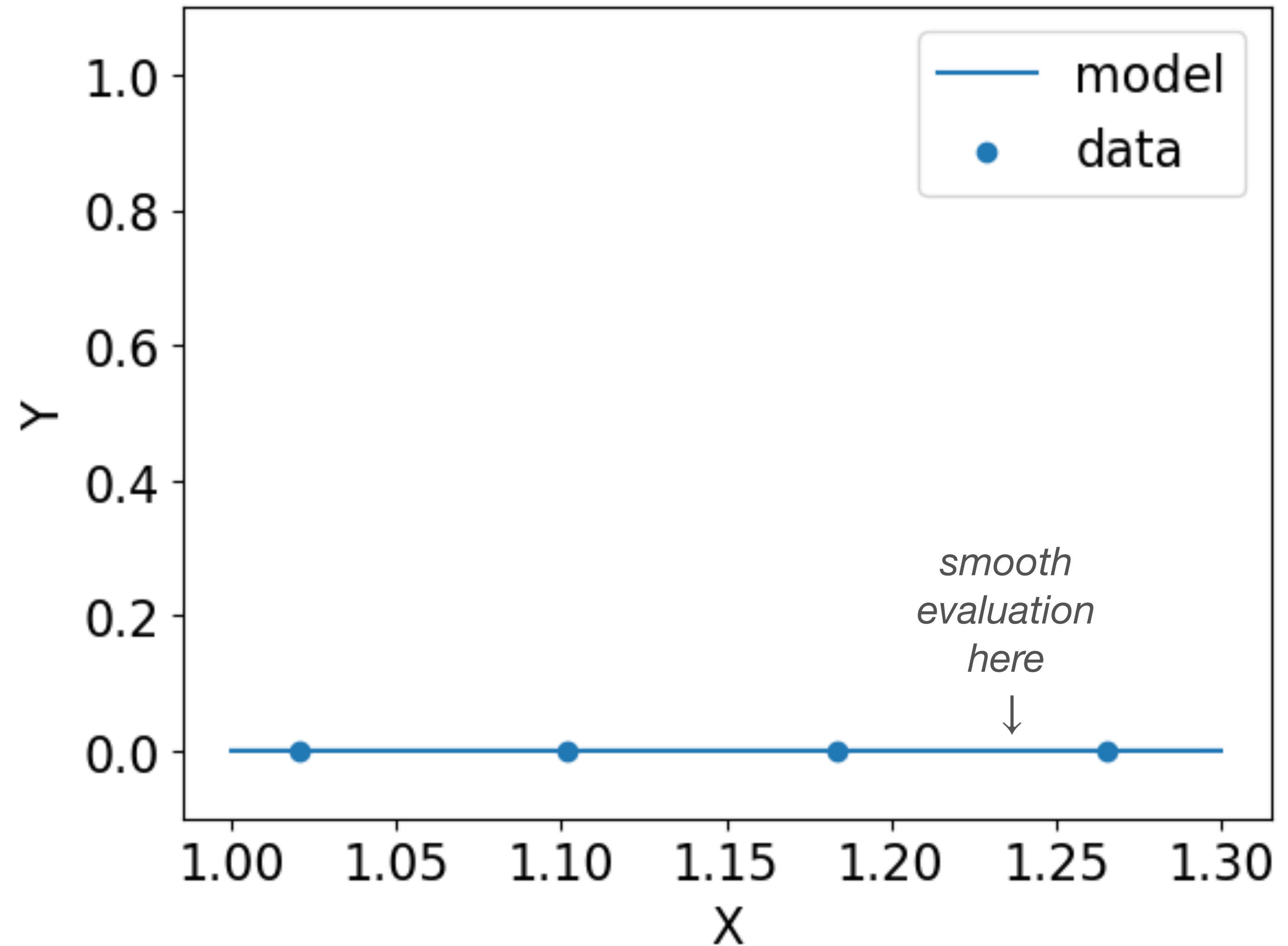
***Wider
optimizes
faster***



- Not just better endpoint, but fewer epochs to train

not obvious why

Generalizes



- Not just better and faster, but generalizes

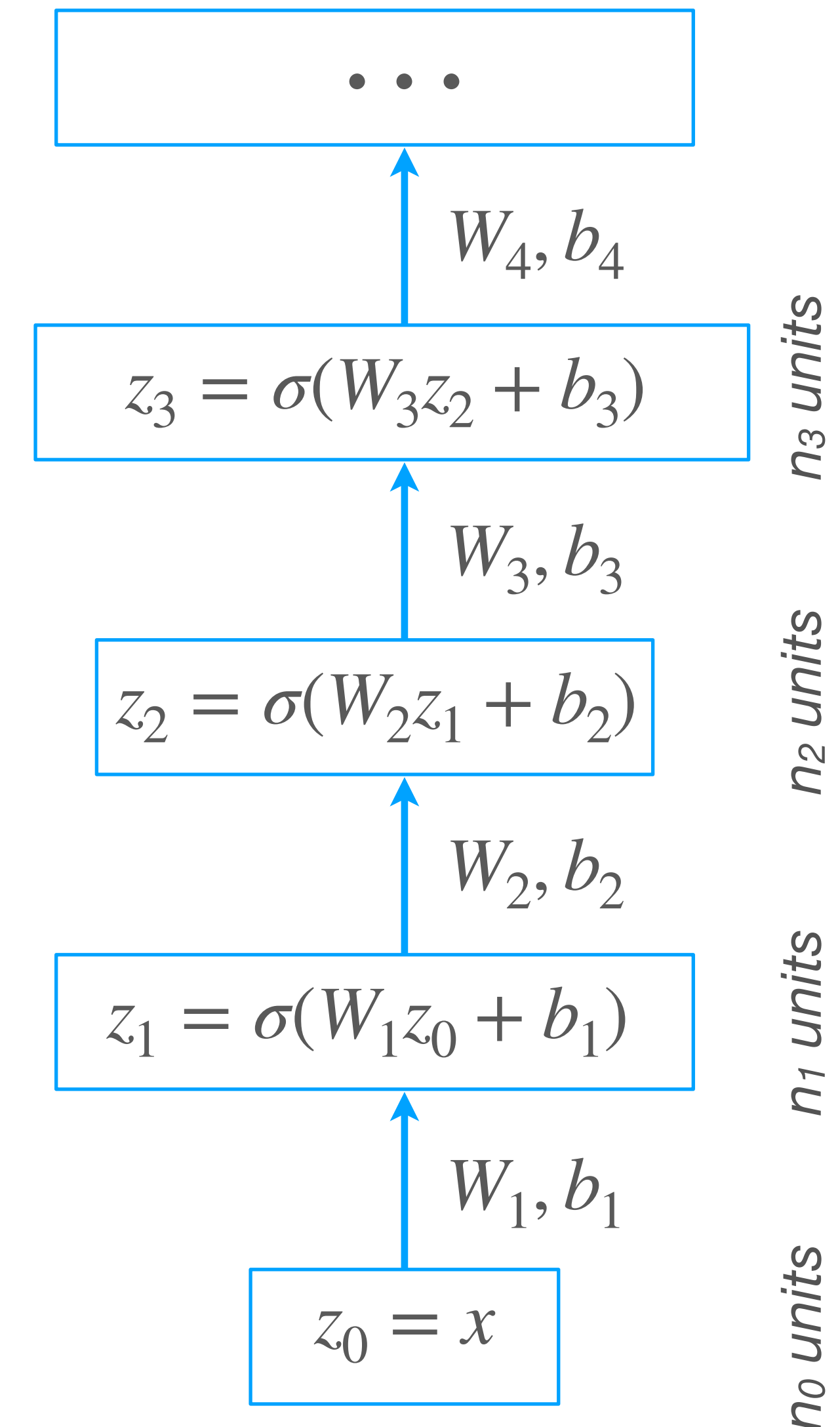
not obvious why

Scaling

- So we want big networks
- But intuition that works on small networks can stop working as we make networks wider and deeper
 - ▶ important to consider *scaling* behavior: how should hyperparameters like learning rate and initialization change as we make network bigger
 - ▶ typically: reasoning about scaling gets hyperparameters into the right general area, still need tuning after
- Vanishing/exploding gradients is one good example of the kind of reasoning we need
 - ▶ choose weight scale for MLP to keep activation approximately constant as depth $\uparrow$

Tools

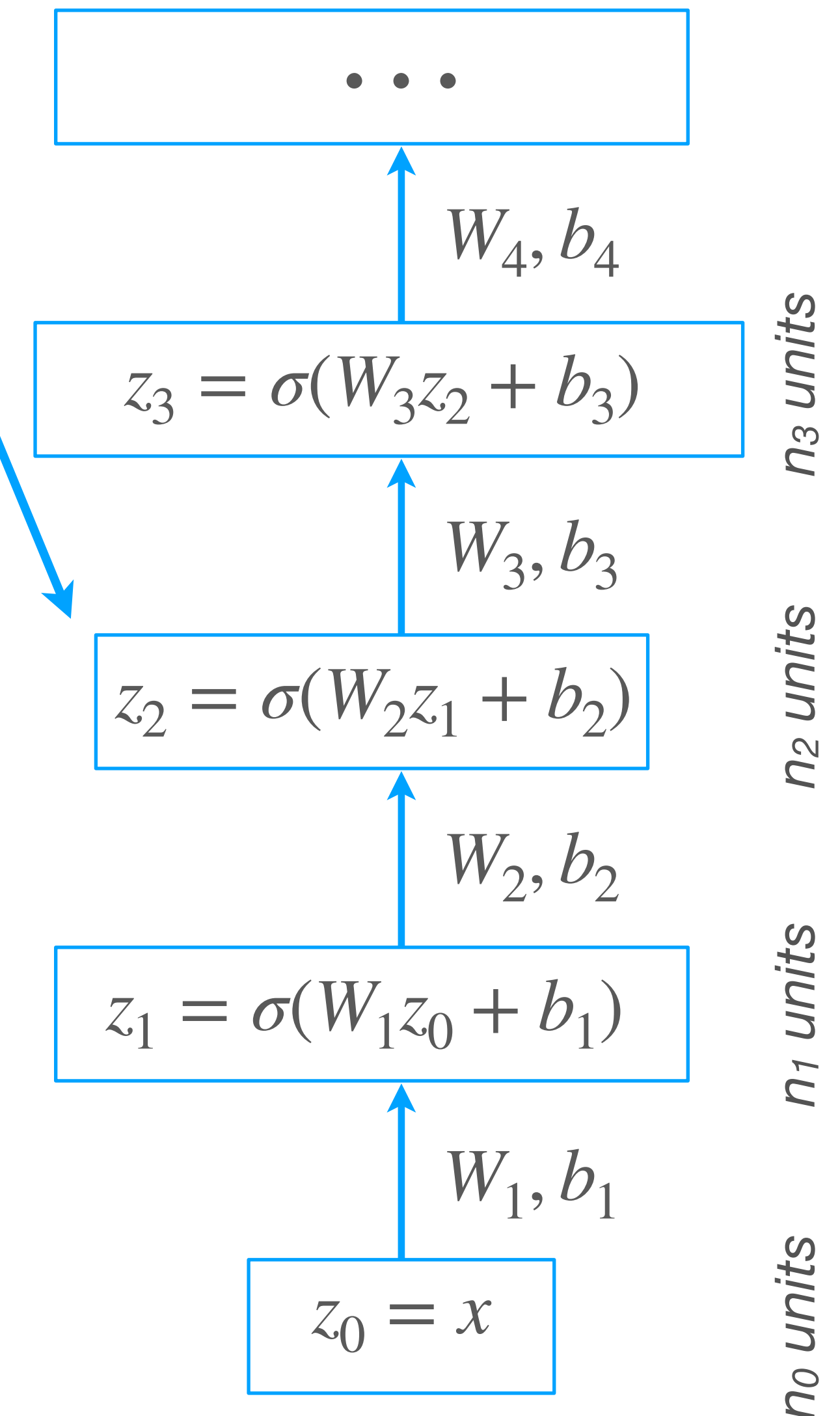
- Uncorrelated variance adds
 - ▶ if we sum up n uncorrelated contributions, each with variance σ^2 , result has variance $n\sigma^2$
 - ▶ therefore standard deviation $\sqrt{n}\sigma$: updates partially cancel
 - ▶ mean of sum is sum of means (e.g., sum of 0-mean terms has mean 0)
- Fan-out and fan-in
 - ▶ a node's **fan-in** is the number of nodes it receives activations from in the previous layer
 - ▶ a node's **fan-out** is the number of nodes it sends activations to in the next layer



Tools

- Uncorrelated variance adds
 - ▶ if we sum up n uncorrelated contributions, each with variance σ^2 , result has variance $n\sigma^2$
 - ▶ therefore standard deviation $\sqrt{n}\sigma$: updates partially cancel
 - ▶ mean of sum is sum of means (e.g., sum of 0-mean terms has mean 0)
- Fan-out and fan-in
 - ▶ a node's **fan-in** is the number of nodes it receives activations from in the previous layer
 - ▶ a node's **fan-out** is the number of nodes it sends activations to in the next layer

nodes in this layer have
fan-in n_1 and fan-out n_3



Width scaling — activations

- Nodes in layer $\ell \geq 1$ have fan-in $n_{\ell-1}$
- Suppose activations in layer $\ell - 1$ have scale $\Theta(1)$
- Input (pre-activation) to a node in layer ℓ is sum of weight $\times$ activation for all nodes in layer $\ell - 1$
 - ▶ with random initialization, these terms are uncorrelated and have zero mean
 - ▶ term variance $\propto$ weight variance σ_ℓ^2
 - ▶ So: node's input has mean zero and scale $\Theta(\sqrt{n_{\ell-1}}\sigma_\ell)$
 - ▶ activation scale depends on nonlinearity but also typically $\Theta(\sqrt{n_{\ell-1}}\sigma_\ell)$ — e.g., sigmoid scales about $\frac{1}{4}$
- So: $\sigma_\ell \propto \frac{1}{\sqrt{n_{\ell-1}}} \rightarrow$ activation scale in layer ℓ is $\Theta(1)$

Width scaling — gradients and activation changes

- “Standard” initialization: $\sigma_\ell = \frac{1}{\sqrt{n_\ell - 1}} \times k_{\text{nonlin}}$ where k_{nonlin} is chosen separately for each nonlinearity
- Keeps activations $\Theta(1)$ in all layers at initialization
- Similar calculations show that:
 - ▶ activation gradient entries are $\Theta(1/\sqrt{n_\ell})$
 - ▶ weight matrix gradient entries are $\Theta(1/\sqrt{n_\ell})$
- After SGD step, activations (if we try the same input $X^{(t)}$ again) change by $\Theta(\eta n_{\ell-1} / \sqrt{n_\ell})$ [skip ℓ dependence]
 - ▶ for hidden layers of similar size n , this is $\Theta(\eta\sqrt{n})$
 - ▶ for output layer (constant width) this is $\Theta(\eta n)$

Width scaling — step size

- Output update scaling $\Theta(\eta n)$ suggests setting $\eta \propto 1/n$ so updates don't explode or vanish with increasing width
- But this choice causes an odd behavior: SGD updates to weights and hidden activations are vanishingly small compared to initialization scale
 - ▶ still adds up to $\Theta(1)$ change in output due to growing fan-in
 - ▶ but the net never moves away from its random initialization
- This setting works — the deep net is effectively a complicated device for computing random features
 - ▶ this behavior is called the **kernel** regime since random features are connected to kernel methods
- But we might want to have a deep net whose hidden activations actually change as we train

Width scaling — maximal updates

- Instead, a clever proposal:
 - ▶ initialize output weights differently: $\Theta(1/n)$ vs $\Theta(1/\sqrt{n})$
 - ▶ use different step sizes on each layer: input $\Theta(n)$, hidden $\Theta(1)$, output $\Theta(1/n)$
- With this initialization and step size, activation values change by $\Theta(1)$ with each SGD step (we learn features)
- Called the ***maximal update*** or μP scheme
- In practice: either something like μP or something like standard but with a tuned (usually larger) learning rate

Depth, weight decay, ...

- Also need to scale other hyperparameters (e.g., weight decay, Adam's β s and ϵ)
- Also need to adjust scaling for each optimizer (e.g., Adam tries to scale all gradients to $\Theta(1)$)
- Also need to scale everything with depth, not just width
- There is not always a consensus on best scaling
- Advice: look up latest, rather than trying to memorize
 - ▶ just be aware: check plots of norms of activations, gradients, updates, etc. vs. width, depth, layer index, etc.
 - ▶ remember that default initialization doesn't account for depth or handle choice of kernel vs. feature-learning
 - ▶ be conscious that things can change with seemingly-small edits (e.g., switching SGD to Adam)
 - ▶ choose architecture to mitigate problems (e.g., residual connections)

Nonlinearity

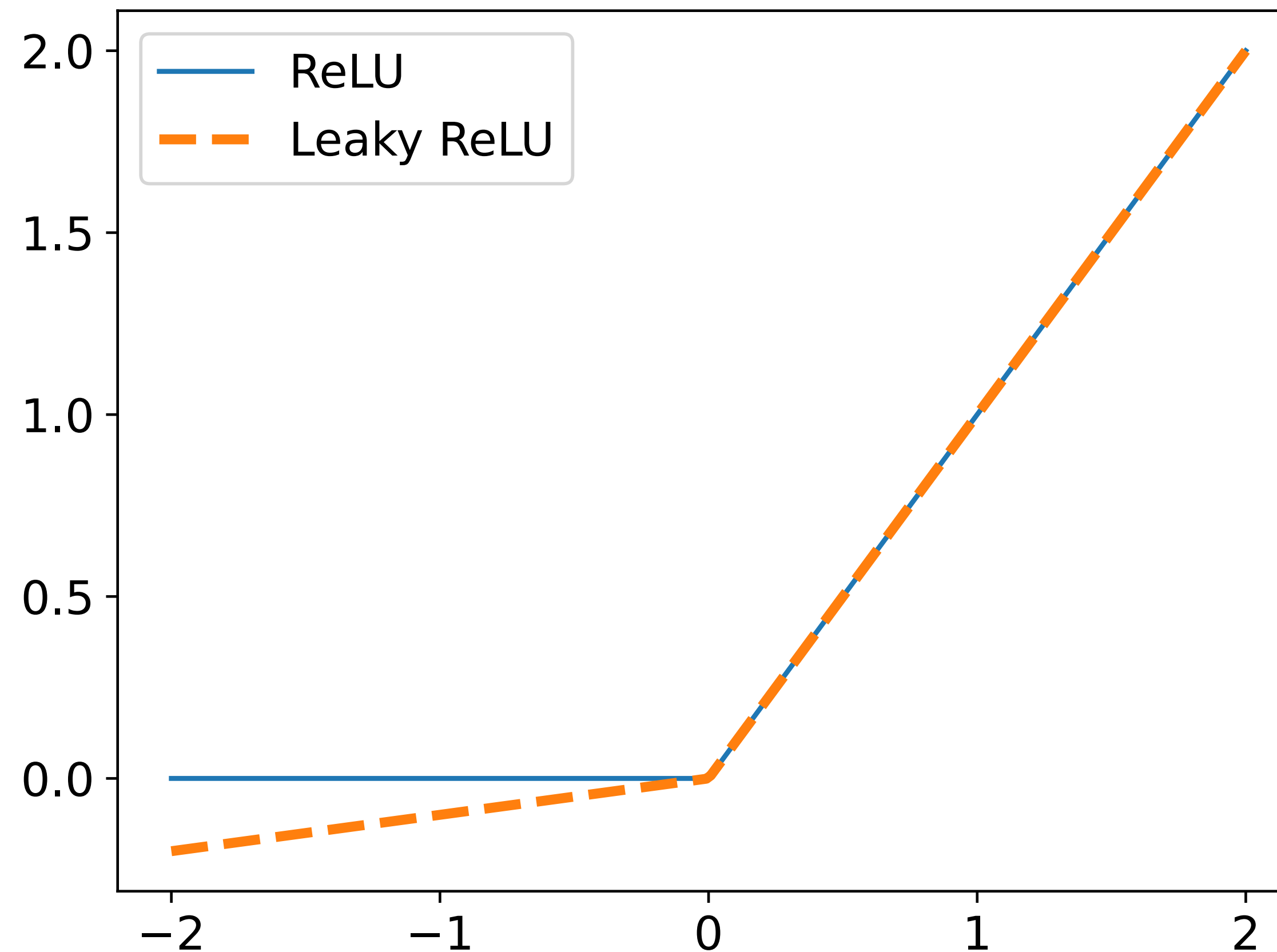
$$\text{softmax}(z_1, z_2) = (\sigma(z_1 - z_2), \sigma(z_2 - z_1))$$

- So far we've seen $\sigma(\cdot)$, $[\cdot]_+$
 - ▶ variant of σ : softmax, $z \mapsto e^z / \sum e^z$, $\mathbb{R}^d \rightarrow \triangle$
 - ▶ should really be “soft argmax,” but who am I to argue
- Many more choices:
 - ▶ leaky ReLU, SILU, GELU, gating, normalization, ...

Dead zones

- Gradient of ReLU $[z]_+$ is 0 for $z < 0$
- OK if this is true for some examples for each unit
- If it's true for all examples at some unit, we're stuck: SGD has no gradient signal to help fix this unit
 - ▶ unit contributes nothing to learned function
 - ▶ might get lucky and SGD noise makes it wander back into life via random changes to input
 - ▶ else, a waste of GPU time and memory

Leaky ReLU



- ReLU can be written $\max(0, z)$
- Leaky ReLU: $\max(\alpha z, z)$ for parameter $\alpha \in (0, 1)$
 - ▶ now we have nonzero gradient even when $z < 0$
- Can fix α or learn it by SGD

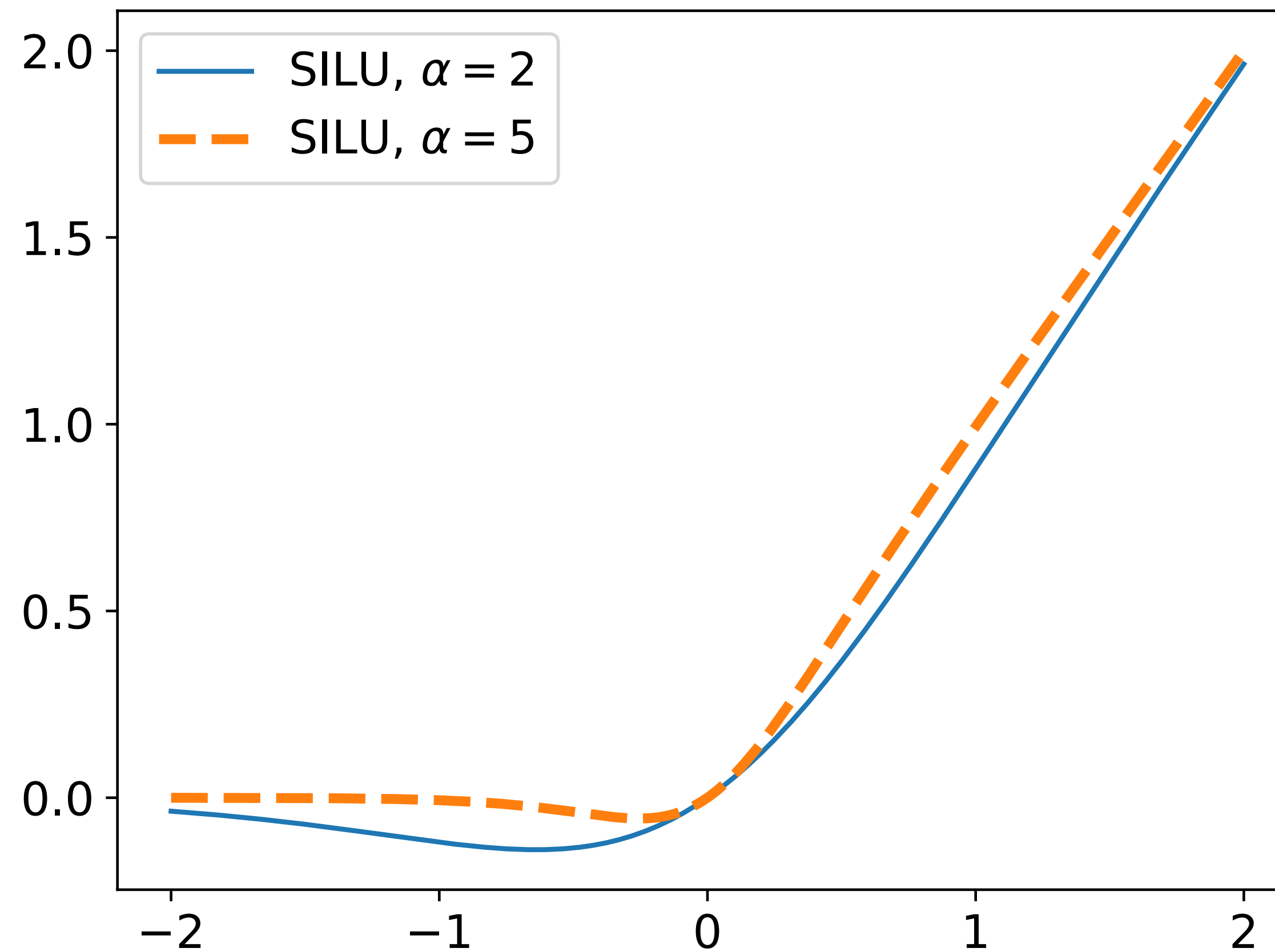
*$\alpha = 0$ yields
original ReLU*

*$\alpha = 1$ yields
identity*

Large negative values

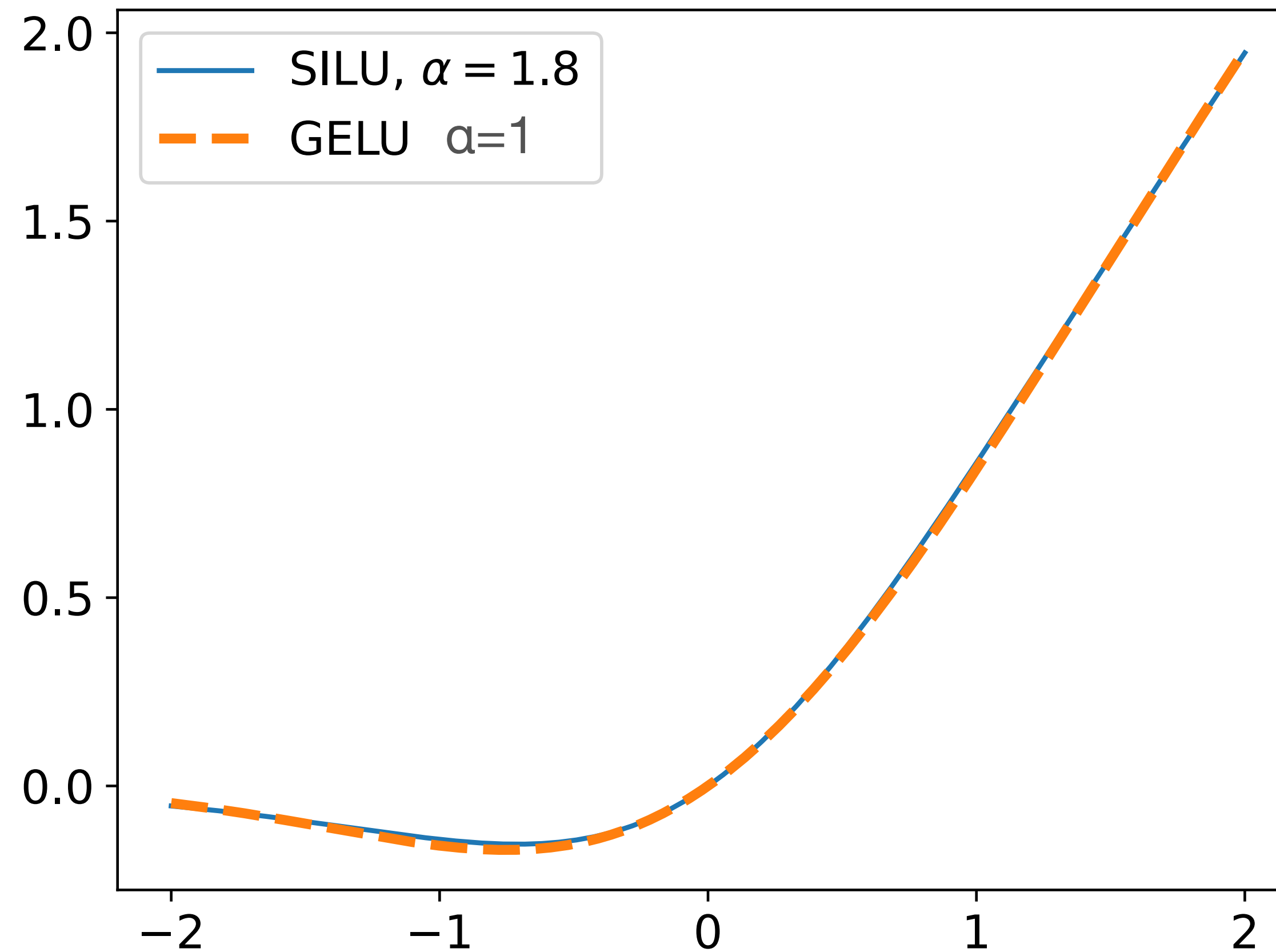
- Part of ReLU's benefit is that it zeros out large negative values of pre-activation z
- Leaky ReLU doesn't do this: if $z \ll 0$, output can still have large absolute value
- Fixes: SILU, GELU
 - ▶ give up monotonicity to recover suppression of $z \ll 0$ while maintaining $f(0) = 0$
 - ▶ can't have all 3 of these properties

SILU



- Sigmoid linear unit = $z \sigma(\alpha z)$ for $\alpha > 0$
 - ▶ also called Swish
 - ▶ approaches ReLU for $\alpha \rightarrow \infty$

GELU

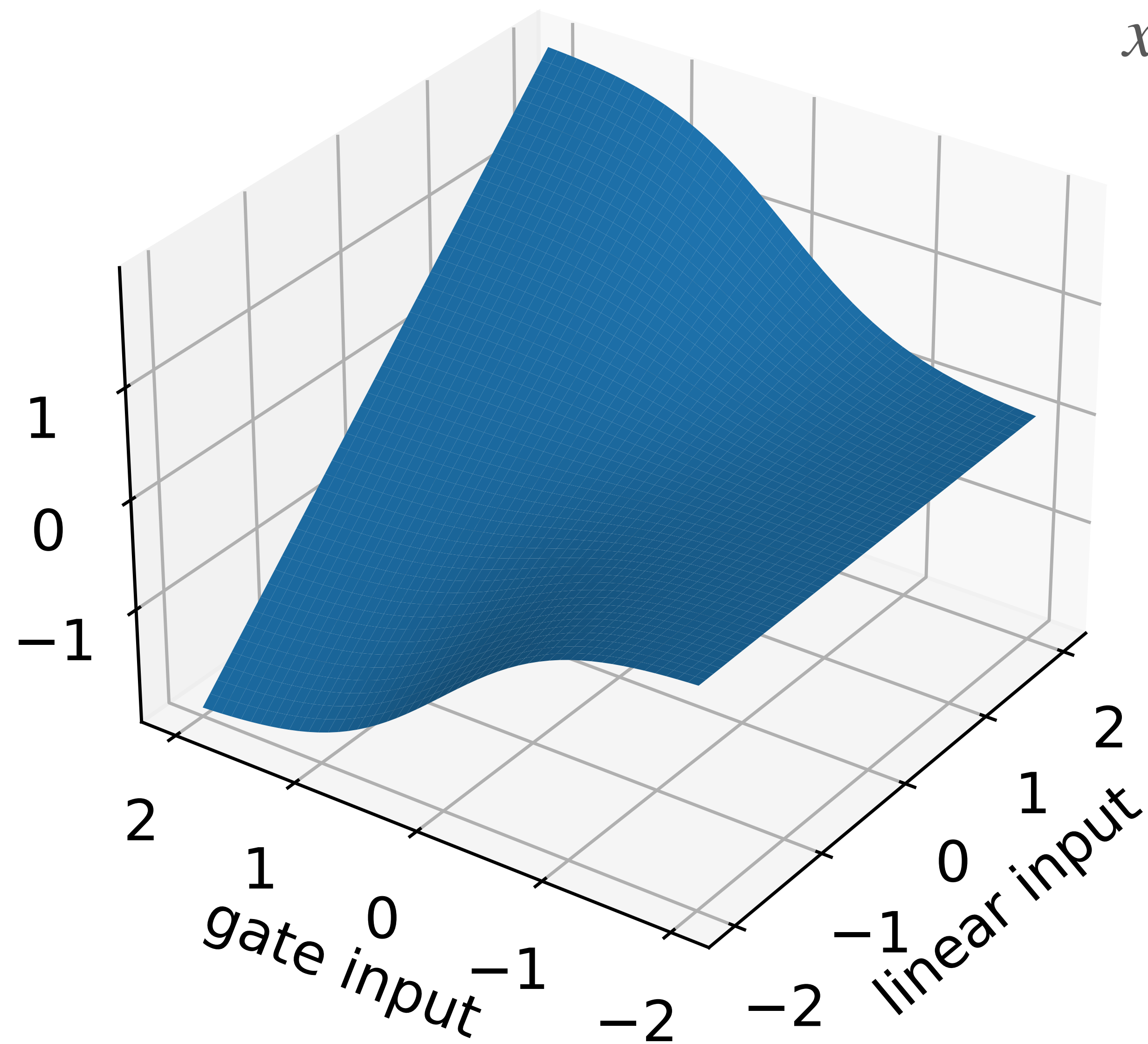


- Can do something like SiLU with any CDF
- Popular: Gaussian CDF $\rightarrow$ GELU
 - ▶ Gaussian error linear unit, since the Gaussian CDF is related to the *error function* $\text{erf}(z)$

Gating

- Can think of ReLU, SILU, GELU as implementing a logical test (a *gate*) on activation $z = Wx + b$
 - ▶ if $z < 0$, suppress output (possibly differentiably)
- What if we want the gate to depend on something other than sign of z ?
- Gated linear unit (GLU):
 - ▶ $(Ux + b) \circ \sigma(Wx + c)$
 - ▶ two separate linear functions of previous layer, second one gates the first
 - ▶ can replace σ with any nonlinearity (Gaussian CDF, Swish, GELU, identity, ...) — SwiGLU is common
- Twice as many parameters (matters for speed, memory) but can counteract by scaling down layer width

GLU



$$x_2 \circ \sigma(2x_1)$$

Normalize

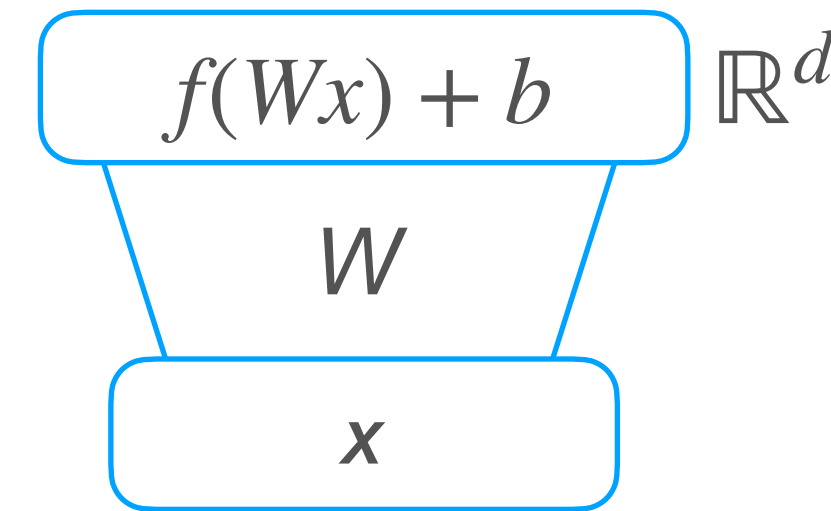
- Typical example layer: $f(Wx + b)$: matrix W , vector x , b , componentwise nonlinear activation f
- Many activations are most interesting near zero
 - ▶ sigmoid's non-constant region
 - ▶ ReLU's derivative discontinuity
 - ▶ ...
- Idea: auto-shift and auto-scale inputs to be in this neighborhood by default
 - ▶ but let the network learn to move away if needed
- Hope: normalization helps generalization
 - ▶ in practice: effects poorly understood, but can be positive

Layer norm

no need for bias weight
or 1s column

tiny fixed number,
to prevent divide-
by-zero

originally: $f(Wx + b)$



- Given $a = Wx \in \mathbb{R}^d$
 - ▶ define $\bar{a} = \frac{1}{d} \sum_{i=1}^d a_i$
 - ▶ define $\sigma^2 = \frac{1}{d} \sum_{i=1}^d (a_i - \bar{a})^2 + \epsilon$
 - ▶ define $a' = \frac{a - \bar{a}}{\sigma}$
- New version of layer, with layer norm: $f(g \circ a' + b)$
 - ▶ new componentwise *gain* parameter $g \in \mathbb{R}^d$ picks scale
 - ▶ existing bias parameter b lets us shift away from zero
 - ▶ but by default (if $g = (1, 1, \dots)^\top$ and $b = 0$), inputs to f have mean zero and variance 1

bias often still
dropped

RMS norm

- Given $a = Wx \in \mathbb{R}^d$
 - ▶ define $\bar{a} = 0$
 - ▶ define $\sigma^2 = \frac{1}{d} \sum_{i=1}^d (a_i - \bar{a})^2 + \epsilon = \frac{1}{d} \sum_{i=1}^d a_i^2 + \epsilon$
 - ▶ define $a' = \frac{a - \bar{a}}{\sigma} = \frac{a}{\sigma}$
- New version of layer, with RMS norm: $f(g \circ a' + b)$
 - ▶ only difference is not to subtract mean as first step
 - ▶ somewhat faster, similar performance

What to normalize?

- Layer norm says the bag of hidden activations for any *single* example should have mean 0, variance 1
- Alternate goal: *each* hidden activation should have mean 0, variance 1 *across* examples
 - ▶ enforcing this property is called *batch norm*
 - ▶ can't measure it with just one example
 - ▶ makes batch norm more complicated to implement

Batch norm

- To the rescue: *minibatch* SGD
- Measure mean and variance in each minibatch
- Given $a_i = Wx_i \in \mathbb{R}^d$ for each example in minibatch B
 - ▶ define $\bar{a} = \frac{1}{|B|} \sum_{i \in B} a_i \quad \bar{a} \in \mathbb{R}^d$
 - ▶ define $\sigma^2 = \frac{1}{|B|} \sum_{i \in B} (a_i - \bar{a})^2 + \epsilon \quad \sigma \in \mathbb{R}^d$
 - ▶ define $a' = \frac{a - \bar{a}}{\sigma}$ ← *componentwise*
- New version of layer, with batch norm: $f(g \circ a' + b)$

contrast w/ layer norm where
mean, variance are scalars

Batch norm: test time

- We don't necessarily have a minibatch available at test time — so we can't compute $\bar{a}$ and σ !
- So, after we're done with SGD on training set:
 - ▶ learn $\bar{a}$ and σ from entire training set
 - ▶ these parameters become part of our model
- Tempting: test in minibatches, compute $\bar{a}$, σ then
 - ▶ why is this wrong?

Exploding gradients

$$z = f(g \circ \frac{Wx - \bar{a}}{\sigma} + b)$$

$$dL = \langle r, dz \rangle \quad dz =$$

- All these normalization operations are differentiable
- When activations are small or large, normalization scales them
 - ▶ → possibility of vanishing or exploding gradients
 - ▶ must use a mitigation if we have normalization layers
 - ▶ most popular: residual connections